

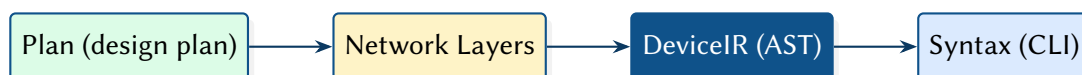
NetCfg: A Declarative Network Configuration Compiler

Formal Methods in Configuration Synthesis

Simon Knight, Ph.D.

March 2026 • Version 4.0.0

Last updated: March 13, 2026



Abstract. NETCFG is a Rust-based network configuration compiler that takes a annotated topology—the “source code” of the network—and compiles it into verified, target-specific device configurations. A companion architectural plan selects which compilation strategies to apply. The compiler processes these inputs through a five-layer pipeline: (1) Intent Transformation, (2) Topological Validation, (3) Lowering to a vendor-neutral Device IR, (4) Target-Specific AST Transformation, and (5) Syntax Serialisation. This report documents the data model, the transformation DSLs, and the AST-based lowering mechanism that decouples architectural logic from physical hardware constraints.

Contents

I ARCHITECTURE & INSIGHTS

1	Introduction	2
2	Conceptual Model	4
2.1	A mental model of the compiler	4
2.2	The network model as a layered graph	5
2.3	Three input documents	8
2.4	Topology provenance	10
2.5	design plans compose the model declaratively	10
2.6	Mini worked example (conceptual)	11
2.7	The select-enrich-validate pattern	12
3	The Network Model	13
3.1	Layers	13
3.2	Per-node state: DeviceContext	13
4	Intent and Policy Expression	14
4.1	The five concerns of a network design plan	15
4.2	From plan to network model: declarative composition	15
4.3	Expressing policy at the plan layer	16
4.4	Reusable design-rule libraries	16
5	The Compilation Pipeline	17
6	Related Work	18

II THE DESIGN PLAN LANGUAGE

7	Structuring the compilation plan	18
8	Selector DSL (NQL Surface Syntax)	19
9	Named groups	20
10	Named objects	21
11	Primitive catalogue	21
11.1	mesh_nodes	24
11.2	provision_ips	24
11.3	build_protocol_layer	24
11.4	allocate_resources	25
11.5	global_pool	25
11.6	global_resource_pool	25
11.7	conditional	25
11.8	custom_primitive	25
11.9	inject_secrets	26
11.10	build_routing_policy	26
11.11	build_access_policy	26
11.12	wasm_plugin	27
11.13	build_prefix_list	27
11.14	build_community_list	28
11.15	generate_safe_bgp_filters	28
11.16	build_zone_policy	28
11.17	build_nat_policy	29
11.18	build_qos_policy	29
11.19	build_vxlan	30
11.20	build_evpn	30

11.21	tag_nodes.....	31
11.22	map_hardware_inventory	31
11.23	assert_state.....	31
12	Design-rule assertions	31
13	Named rules (NQL macros)	33
14	Blast radius policies	33
15	Target-specific transforms	34
III	ENGINE IMPLEMENTATION (DEEP DIVE)	
16	Pipeline Internals	34
16.1	Layer 1: Intent Transformation	34
16.2	Layer 2: Topological Assertions	35
16.3	Layer 3: DeviceIR and the Mapping DSL	35
16.4	Layer 4: Target-Specific Transform (TSDM)	36
16.5	Layer 5: Syntax Serialisation.....	36
17	Core Mechanisms	37
17.1	Declarative IPAM and Resource Allocation	37
17.2	Protocol overlay construction (build_protocol_layer).....	38
17.3	Configuration diffing (DiffEngine)	39
17.4	AST Transformations (TSDM Layer)	40
18	CLI Reference	40
18.1	netcfg generate.....	41
18.2	netcfg validate.....	41
18.3	netcfg plan.....	42
18.4	netcfg inspect.....	42
18.5	netcfg ci-run.....	42
18.6	netcfg import.....	43
18.7	netcfg lint.....	43
18.8	netcfg fmt.....	43
18.9	netcfg impact.....	43
18.10	netcfg report.....	43
18.11	netcfg export-clab.....	44
18.12	netcfg sync.....	44
19	End-to-End Worked Example	44
19.1	Input: design plan	44
19.2	Stage 1: Parse and validate	45
19.3	Stage 2: Shadow topology.....	45
19.4	Stage 3: Primitive execution	45
19.5	Stage 4: Mapping evaluation.....	45
19.6	Stage 5: Render	46
19.7	Stage 6: Atomic write	46
20	Error Model	46
21	Editor and Service Integration	47
21.1	Language Server (LSP)	47
21.2	REST API Microservice.....	47
22	Limitations	48
22.1	edge_properties not applied (mesh_nodes).....	48
22.2	Mapping AST inspection not implemented	48
22.3	Shadow validation is a placeholder	48

22.4	No intra-primitive rollback	48
22.5	generate CLI bypasses mapping document	49
22.6	No incremental compilation	49
22.7	No runtime verification or drift detection	49
22.8	Shadow topology cloning cost	49
22.9	WASM plugin is experimental	49
22.10	Mapping document limitations	49
22.11	NSoT push not implemented (netcfg sync push)	50
23	Conclusion	50
24	Future Work	50

1 Introduction

Large-scale network configuration is a compilation problem.

Shenker [1] diagnosed the problem: networking adds protocols rather than abstractions. This produces a “bag of protocols” with no composable interfaces. He proposed three abstractions to decouple intent from mechanism: a forwarding interface, a global network view, and a specification language.

Gill et al. [2] quantified the cost: misconfiguration causes most customer-impacting incidents in Microsoft data centres, usually triggered by operator error. As data centres grow to millions of lines of config, the need for automated, intent-driven synthesis increases. Yet most automation remains bespoke and non-transferable.

Recently, large-scale topology systems [3] have introduced *multi-abstraction-layer topology* modelling. A shared representation spanning physical, logical, and design layers unifies network operations. The key insight—that “almost all other network-management data either derives from its topology, constrains how to use a topology, or associates resources with specific places”—motivates NETCFG’s graph-centric design.

Meanwhile, model-driven management via OpenConfig [4] and YANG [5] has standardised vendor-neutral *per-device* data schemas, but these models do not capture cross-device topology relationships or support graph-wide operations like IP allocation.

Insight:
Manual CLI entry is the “assembly language” of networking; NETCFG provide the high-level language and compiler.

: Hardware Independence

Architectural logic must lower to a vendor-neutral Intermediate Representation (DeviceIR) before any target-specific syntax is generated. This decouples policy intent from physical ASIC constraints.

What NETCFG is

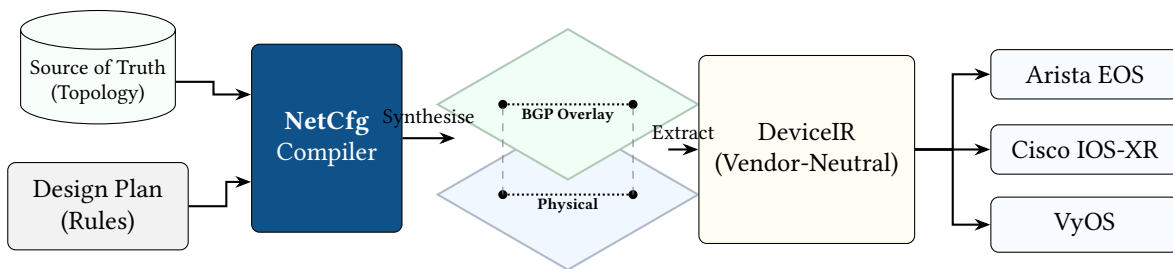


Figure 1. NetCfg acts as the compilation bridge between Source of Truth systems and physical network hardware. Powered by its declarative query language, it synthesises the physical inventory and design rules into logical **Network Layers** (e.g., BGP, OSPF). This allows operators to validate their architectural intent *before* extracting the Vendor-Neutral Intermediate Representation (DeviceIR).

NETCFG is a command-line tool and Rust library that compiles an *annotated topology* into per-device configuration files. The annotated topology is the primary input: a graph of devices and links enriched with semantic properties—roles, sites, address pools, protocol flags, trust levels—that encode the operator’s intent at a network-wide level. A companion *design plan* selects which compilation strategies (meshing patterns, allocation policies, assertion rules) to apply when deriving device-level state from those annotations. Optionally, a *mapping* document describes how to extract vendor-specific configuration stanzas from the enriched topology. The compiler produces deterministic, diffable, auditable configuration artefacts for every device in the network.

NETCFG replaces the manual “glue” layer between source-of-truth systems (NetBox [6], Nautobot [7]), automation frameworks (Ansible [8], Nornir [9]), and network modelling tools (Batfish [10]). It

treats network configuration as a *compilation problem*: topology intent is compiled through a typed pipeline, not interpolated through ad-hoc templates.

What NETCFG is *not*

To understand the architecture, define its boundaries:

- **It is not a Source of Truth.** It does not store inventory or manage IPAM databases. It *consumes* data from systems like NetBox and projects it into a layered graph.
- **It is not a Deployment Engine.** It does not SSH into routers or push configuration via NETCONF/gNMI. It *compiles* the declarative artifacts that tools like Ansible then deploy.
- **It is not a Data-Plane Verifier.** It does not simulate packet flow like Batfish. It proves *structural intent* (e.g., “Are these zones isolated in the model?”) rather than simulating routing tables.

Where OpenConfig standardises the *schema* of device state, NETCFG standardises the *derivation* of that state from topology intent. Earlier systems provided a shared topology *representation*; NETCFG provides a topology *compiler* that transforms intent into per-device configuration.

Design philosophy

Three principles explain every design decision in this document:

1. **Lowering over templating.** The operator’s intent is lowered through successive abstraction stages, mirroring the multi-stage lowering architecture of LLVM [11]. LLVM lowers high-level source through a language-independent IR (LLVM IR) to target-specific machine code; NETCFG lowers topology intent through a vendor-neutral DeviceIR to platform-specific CLI syntax. The analogy is precise: LLVM’s frontend/IR/backend separation corresponds to NETCFG’s design plan/DeviceIR/template separation. Hardware quirks and vendor-specific logic are handled via structured AST transformations, not ad-hoc string concatenation.
2. **Separation of what from how.** The annotated topology declares *what* the network contains—devices, links, and their semantic properties. The design plan declares *which* design rules to apply when compiling that topology (meshing style, allocation strategy, assertion constraints). The mapping and templates declare *how* to render the enriched model into per-device configuration. Platform-specific rules are injected via imports: [‘hardware-lowering.yaml’] and isolated behind when: `device_os == ... predicates`. The same topology and design plan can produce Cisco IOS, Arista EOS, or Juniper JunOS output by swapping only the mapping and templates.
3. **Topology as the single source of truth.** Topology is the root from which all other network-management data derives. NETCFG treats the graph as the canonical representation. IP addresses, protocol sessions, policy rules, and security zones are *computed* from topology relationships, not declared independently.

Audience

This document is intended for:

- Engineers integrating NETCFG into a network automation pipeline.
- Contributors to the NETCFG and NTE codebases.
- Researchers evaluating graph-based approaches to network configuration.

Familiarity with YAML, basic graph theory, and IP networking is assumed. Experience with Rust is helpful but not required for the API sections.

Companion report: NTE-TR-001

NETCFG implements the compilation leg of the Write-to-Production (W2P) strategy defined in NTE-TR-001 §Core Architectural Insights. NTE provides the graph substrate; NETCFG compiles annotated topologies into verified device configurations. NETCFG operates exclusively in NTE’s Strict Mode (design-authority, schema-validated compilation).

How to read this document

This report is organised into three parts. **Part I** (Architecture & Insights) introduces the conceptual model, the network data model, intent and policy expression, and the compilation pipeline—purely conceptual, with no implementation detail. **Part II** (The Design Plan Language) specifies the declarative surface: plan structure, selector syntax (NQL), named groups and objects, the primitive catalogue, assertions, blast-radius policies, and target-specific transforms. **Part III** (Engine Implementation) covers the pipeline internals, core mechanisms (IPAM, protocol overlay, DiffEngine, TSDM), the CLI, an end-to-end worked example, the error model, editor integration, and current limitations.

2 Conceptual Model

This section explains the three ideas underpinning every NETCFG design plan: (1) the network model is a *layered graph*; (2) design plans *compose* the model declaratively, layer by layer; (3) every primitive follows the same *select* → *enrich* → *validate* pattern. Understanding these concepts makes the formal data model (§3) and DSL syntax (§II) easier to absorb.

Keep the mental model, postpone the mechanisms. In this section, treat NETCFG as a compiler operating over a graph: selectors identify where in the graph to act, primitives add derived state, and assertions act as quality gates. The implementation details (Rust structs, NQL query compilation, DataFrames) exist to make this model fast and safe, but they are not required to understand the flow.

2.1 A mental model of the compiler

NETCFG strictly separates **Engine Knowledge** (Rust core, graph mechanics) from **Domain Knowledge** (DSL, vendor rules, templates). No network design logic is hardcoded in the engine. The compiler takes an annotated topology and derives device-level state until it can render deterministic, diffable configuration.

The funnel implements the **Configuration Synthesis** architecture (§1): Topology Enrichment → Policy Validation → Vendor-Neutral Extraction → Hardware Adaptation → CLI Generation. Figure 2 shows the funnel; Table 1 summarises the stage outputs.

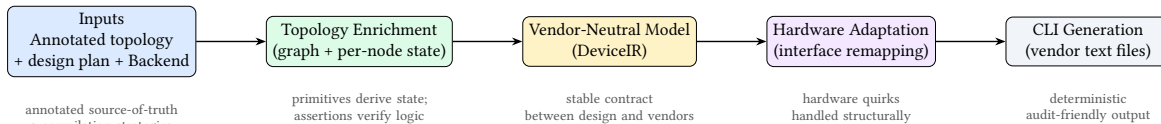


Figure 2. The Configuration Synthesis Funnel. The primary input is the annotated topology; the design plan enriches the layered graph; the compiler extracts a vendor-neutral DeviceIR, adapts it to physical hardware constraints, and generates vendor CLI syntax.

Table 1. Stage outputs (conceptual artefacts). These conceptual stages map to the synthesis pipeline: Enrichment = Layer 1, Validation = Layer 2, Extraction = Layer 3, Adaptation = Layer 4, Generation = Layer 5.

Stage	Output artefact	What it represents
Input	Annotated topology + design plan + backend package	Source-of-truth graph, compilation strategies, and a reusable backend.
Enrichment	Enriched layered model	A single graph containing physical and logical layers, with per-node derived state.
Extraction	DeviceIR (Vendor-Neutral)	Vendor-neutral, structured representation of per-device intent.
Adaptation	Target Device Model (TSDM)	Platform-specific structure (interface naming, feature support, syntax constraints).
Generation	Text configuration files	Deterministic device configs suitable for diff, review, and deployment.

2.2 The network model as a layered graph

NETCFG represents the network as a graph of *nodes* (devices, protocol instances) and *edges* (links, peering sessions). The graph is organised into *layers*: the *physical layer* contains routers, switches, and cables; each *protocol layer* (BGP, OSPF, etc.) contains a parallel set of nodes—one per device participating in that protocol—linked back to their physical origin.

What a “layer” means. A layer is a named, self-contained subgraph representing one plane of intent. The physical layer captures hardware and cabling. Protocol layers capture logical relationships (adjacencies, sessions, overlay bindings) that are *hosted* on those physical devices. Layers are not “views” over the same nodes; each layer has its own node set. The compiler represents both “device A is cabled to device B” (physical) and “A peers with C via a route-reflector” (BGP) even when those relationships do not coincide.

A physical router **A** thus has a corresponding BGP node **A'** and (optionally) an OSPF node **A''**. Each protocol node carries a *parent mapping* that traces it back to the physical device that hosts it. This cross-layer traceability is how IP addresses allocated on the physical layer flow into BGP configuration without explicit wiring.

Parent mapping and traceability. Every non-physical node stores the identifier of its host physical device (`_source_id` in `data_json`). This forms a many-to-one mapping from overlay nodes to physical nodes: multiple protocol instances (BGP, OSPF, EVPN, IPsec) may all point back to the same physical router. The parent mapping is the only cross-layer link.

Figure 3 illustrates this. The physical layer shows three interconnected devices. The protocol layers above contain corresponding protocol nodes. Dashed vertical arrows show the parent mapping tracing back to the physical hardware.

Insight:
Because overlays never directly reference other layers, the compiler can clone, transform, and validate each layer independently while preserving an unbroken “why” trail back to the physical topology.

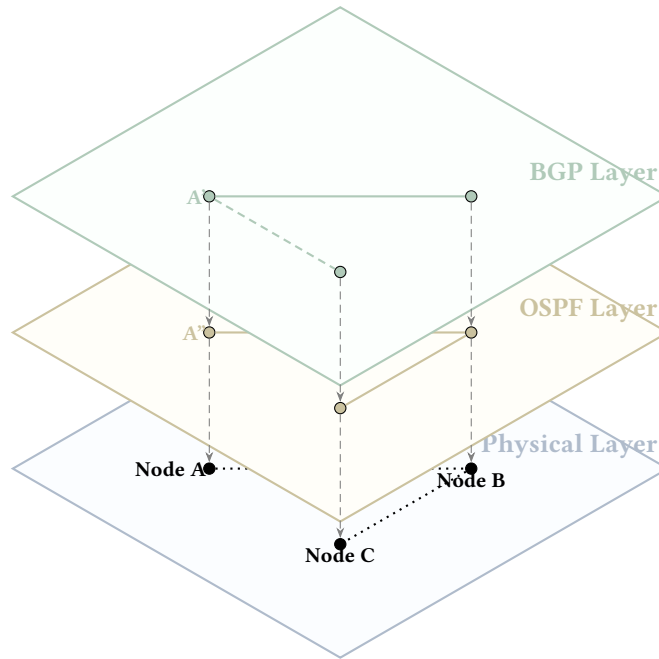


Figure 3. Multi-layer Topology Projection. Powered by the NETCFG graph engine, each routing protocol lives in its own logical sub-graph. The BGP layer establishes a logical peering session between A' and C' (dashed), even though A and C are not physically adjacent. Vertical dashed lines represent the “parent” mapping tying the logical nodes down to the physical hardware.

Where facts live (and how they flow across layers). As a rule of thumb, facts originate in the layer where they are naturally defined: (1) *link and interface facts* (peer interfaces, link subnets, L2/L3 addressing) originate on the physical device nodes; (2) *protocol intent* (neighbour sets, areas, timers, policy bindings) originates on the protocol-layer nodes.

When a protocol node needs a physical fact (e.g., a neighbour IP), it obtains it by following its parent mapping to the host physical node and reading the already-derived fields there. This is how “allocate on physical, consume in overlay” works without bespoke wiring.

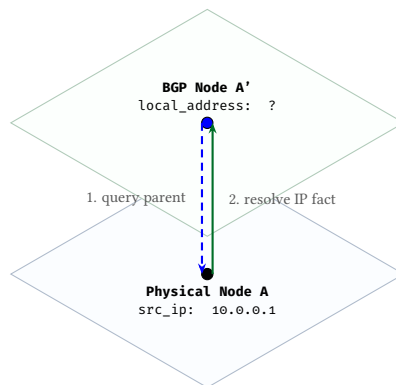


Figure 4. Cross-layer state resolution. Protocol overlays do not maintain redundant state. They resolve physical facts (like allocated IPs) on-the-fly by querying their host physical node via the `_source_id` parent mapping.

Edges within a layer represent direct relationships: physical cables, peering sessions, or policy bindings. Edges *never* cross layers; the parent mapping is the sole mechanism for cross-layer traceability. All mutable state—IP addresses, protocol parameters, security zones—lives on *nodes*, not on edges. Edges exist solely to express relationships between nodes.

Why node-centric state matters. Configuration is ultimately rendered per device. By ensuring every piece of renderable information is attached to nodes (and by keeping edges as pure relationships),

the mapping stage can generate a device's configuration by reading exactly one node's accumulated DeviceContext. Edges still drive computation (e.g., "allocate a /30 per link"), but the results of that computation are always written back onto the endpoint nodes.

Layering as a compilation mechanism. Layers are created and populated by primitives. For example: `mesh_nodes` adds physical edges; `provision_ips` annotates the physical nodes with link addresses; `build_protocol_layer` clones a selected set of physical nodes into a protocol layer and deep-merges a protocol overlay into the clones. Subsequent primitives can then select either physical nodes or protocol nodes by `layer==...`

Note

Design note: protocol edges are logical, not physical. Protocol-layer edges represent intent-level relationships (e.g., "BGP session exists between A' and C" or "policy binds to this adjacency"). They may be derived from physical links, but they are not required to mirror the physical layer's connectivity.

Kinds of layers (a practical taxonomy)

Layers are intentionally lightweight: they let the model hold multiple simultaneous views of the same network without conflating their semantics. In practice, most NETCFG design plans use a small, repeatable taxonomy:

1. **Physical layer.** Nodes represent devices; edges represent physical links. Typical state: interface naming, link addressing, management reachability, hardware inventory.
2. **Protocol layers (routing/switching overlays).** Nodes represent a device's participation in a protocol (one node per device per protocol); edges represent logical adjacencies or sessions. Typical state: neighbours, timers, areas/ASNs, policy attachments, protocol feature flags.
3. **Policy and service layers (optional).** Some organisations model policy and services as dedicated layers when the relationships are graph-shaped (e.g., security-zone bindings, NAT relationships, telemetry subscriptions). Others keep these as node metadata attached to the physical or protocol nodes. The design rule is the same either way: *renderable state stays on nodes; edges express relationships*.

This taxonomy is not prescriptive: the point is that layering preserves separation of concerns while keeping every derived fact traceable back to the physical topology via the parent mapping.

Design rules that build overlays

Layering alone provides representation; *design rules* provide synthesis. In NETCFG, design rules are implemented as primitives that read from the current model (often the physical layer) and then materialise overlay structure and per-device stanzas. They are the bridge from plan topology to protocol overlays.

Two common classes of design rule are:

1. **Overlay construction rules.** `build_protocol_layer` clones selected physical nodes into a protocol layer, records the parent mapping (`_source_id`), and deep-merges protocol defaults into the clones. With `clone_underlying: true`, it also seeds the overlay with a derived copy of the physical connectivity.
2. **Session synthesis rules.** Protocol session primitives (e.g. `build_bgp_session`, `build_ospf_session`, `build_isis_session`)¹ consume the overlay graph and emit configuration stanzas on the participating nodes. These rules are where operator intent becomes concrete neighbours, interfaces, and per-peer parameters.

¹Session synthesis primitives are planned but not yet in the catalogue; the current workflow uses `build_protocol_layer` with a config overlay to achieve the same result.

Insight:
The operator does not "mesh in the BGP layer" manually. They declare intent; the design rules materialise the overlay relationships and the resulting per-device stanzas in the appropriate layer.

2.3 Three input documents

An operator supplies three documents to the compiler:

1. **Annotated topology** — the “source code” of the network. A graph of devices (nodes) and links (edges), where each node carries rich semantic annotations: `hostname`, `role`, `site`, `device_os`, trust levels, address pools, protocol flags, and any other properties the operator considers part of the network’s design intent. The topology may be hand-authored (YAML/JSON), imported from a network inventory system via `netcfg import` (§18.6), or pulled live from an NSoT such as NetBox [6] via `netcfg sync pull` (§18.12). These annotations are the data that the compiler’s selectors (e.g. `nodes[role=='core']`) and primitives operate on. The richer the annotations, the more the compiler can derive automatically. Ideally, the topology captures *all* network-wide design intent; the design plan then reduces to a thin set of compilation strategy choices.
2. **design plan** — *which* compilation strategies to apply. A declarative specification declaring layers of primitives that select subsets of the annotated topology and derive new state: meshing patterns, IP allocation pools and strategies, protocol overlay construction, policy rules, and assertion constraints. The design plan does not duplicate what is already in the topology; rather, it names the *design rules* that transform topology-level annotations into device-level state. The relationship is analogous to a traditional compiler: the annotated topology is *source code* and the design plan is a *compilation profile* (flags such as `-O2` or target architecture). The source code carries the program’s semantics; the compilation profile selects *how* those semantics are lowered. Similarly, the topology carries network intent; the design plan selects which primitives lower that intent into device-level state.
3. **Target backend** — *how* to turn the enriched model into per-device configuration. In practice this is a small directory containing (i) target-specific transforms that lower vendor-neutral intent into platform-specific structure, and (ii) templates that serialise that structure into concrete CLI or configuration syntax. Most organisations author this once per platform and reuse it across sites.

Note

Where does intent live? The boundary between topology annotations and design plan declarations is a design choice that operators can tune. At one extreme, the topology carries only physical inventory (hostnames and cabling) and the design plan encodes all design rules. At the other, the topology carries rich intent (address pools, protocol parameters, zone memberships) and the design plan is a thin strategy selector. NETCFG supports both styles; the general trajectory is toward richer topologies with thinner design plans, because this keeps intent closer to the source-of-truth and reduces the coupling between the design plan and a specific topology’s node schema.

2.3.1 Canonical annotation vocabulary

Table 2 lists the well-known annotation keys that NETCFG recognises on topology nodes. Operators may attach *any* key-value pair; the keys below receive special treatment from selectors, primitives, or the rendering engine.

Table 2. Well-known topology annotation keys. Keys marked with $\star$ are computed by the selector engine and need not appear in the topology file. All other keys are authored by the operator (or imported via `netcfg sync pull`).

Key	Type	Consumed by
hostname	string	All primitives (node identity)
role	string	Selectors (<code>nodes[role=='spine']</code>)
site	string	Selectors, multi-site design plans
device_os	string	Template selection, vendor transforms, QoS constraints
management_ip	string	Inventory; passed through to templates
zone	string	<code>build_zone_policy</code> , zone-based assertions
platform	string	Hardware inventory transforms
vrf	string	<code>provision_ips</code> (routing context separation)
disabled	bool	Selectors (exclude nodes from processing)
<i>Computed selector variables $\star$</i>		
id	integer	Node identifier in the graph
layer	string	Current graph layer (e.g. <code>physical</code> , <code>bgp</code>)
degree	integer	Number of adjacent edges
is_leaf	bool	True when <code>degree == 1</code>
neighbors	list	Adjacent node identifiers

Any key not listed above is stored in the node’s metadata map and remains accessible to selectors and templates. For example, an operator might annotate nodes with `trust_level: high` or `loopback_pool: 10.255.0.0/24`; these values are available to `NETCFG` queries and MiniJinja templates without any schema changes.

2.3.2 Thin design plan vs. rich topology

The boundary between topology annotations and the architectural plan is a design choice. At one extreme, the topology carries only physical inventory (hostnames and cabling) and the plan encodes all design rules. At the other, the topology carries rich intent (address pools, protocol parameters, zone memberships) and the plan is a thin strategy selector.

`NETCFG` supports both styles, but the general trajectory is toward *richer topologies with thinner plans*. This keeps intent closer to the source-of-truth and reduces the coupling between the design rules and a specific topology’s node schema.

In a “fat plan” style, the design rules must name individual nodes and inject properties directly—tightly coupling the plan to a specific topology size. In a “thin plan” style, the topology carries its own identity (roles, sites, platforms), and the design rules operate purely on *semantic scopes* (e.g., “all edge routers”). The same architectural plan works unchanged if a fourth router is added to the topology.

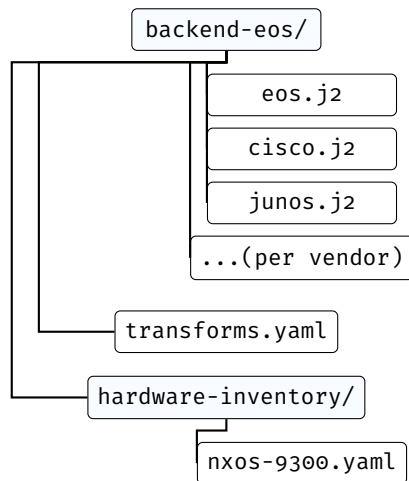


Figure 5. Target backend package structure. Templates render per-vendor syntax; `transforms.yaml` declares TSDM lowering rules; `hardware-inventory/` maps chassis models to slot layouts. Most organisations author this once per platform and reuse it across sites.

The annotated topology is the primary authoring surface: it captures the network’s identity, structure, and design intent. The design plan selects compilation strategies; the mapping document and templates are typically authored once per vendor platform and reused across sites.

2.4 Topology provenance

The annotated topology can enter `NETCFG` through three channels, each preserving the node annotations that selectors and primitives operate on:

1. **Hand-authored YAML/JSON.** The operator writes the topology file directly, placing each node’s annotations under a `properties:` block. This is the most explicit channel and is common during greenfield design or lab work.
2. **Import from existing configurations** (`netcfg import`, §18.6). Parses a directory of device configurations (or a device inventory file) and produces a topology archive (`.nte`), inferring hostnames, links, and device platforms from the configuration data. Operators then enrich the imported topology with additional annotations (roles, sites, trust levels) before compiling.
3. **NSoT synchronisation** (`netcfg sync pull`, §18.12). Pulls a live topology from a network source of truth such as NetBox [6] via its GraphQL API, mapping inventory fields to topology annotations. This channel keeps the annotated topology in sync with the production network.

Regardless of channel, the result is the same data structure: a directed graph with a JSON property blob per node. Primitives and selectors are agnostic to provenance—a topology imported from NetBox is compiled identically to one hand-authored in YAML.

2.5 design plans compose the model declaratively

A design plan organises primitives into ordered *layers*. Each layer declares a set of primitives; each primitive selects a subset of the graph and enriches the nodes it touches. The result is a progressively richer model, as Figure 6 illustrates.

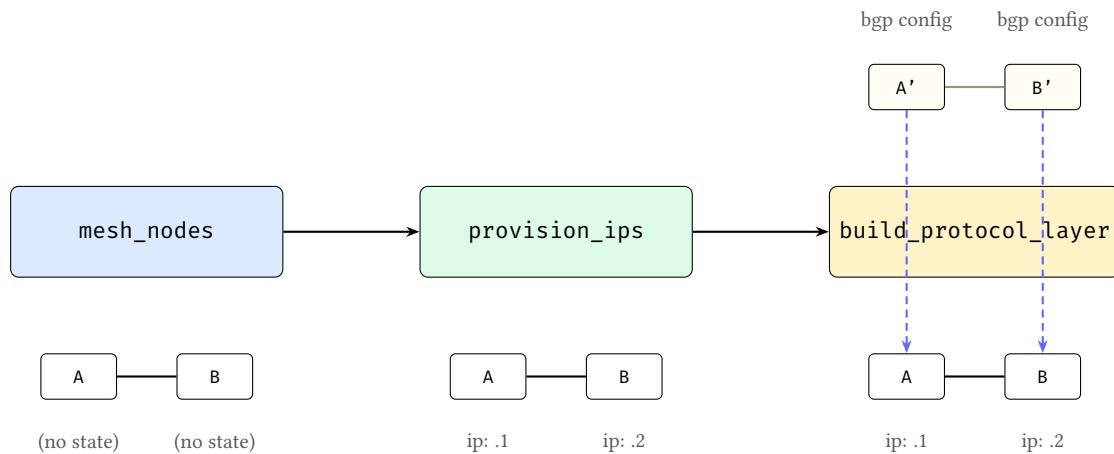


Figure 6. Progressive model enrichment. Left: `mesh_nodes` establishes edges between bare nodes. Centre: `provision_ips` annotates nodes with IP addresses. Right: `build_protocol_layer` clones nodes into a protocol layer and deep-merges a protocol overlay into the clones; dashed arrows show the parent mapping back to the physical devices. At each stage the model grows richer; no primitive needs to know what others have done.

Three properties make this composition model work:

1. **State lives on nodes, not edges.** Edges define relationships—physical links, peering sessions, policy bindings. All mutable, renderable state is written to the node at each end of an edge. Edges may carry static relationship metadata (e.g., labels), but do not carry configuration-bearing state.
2. **Primitives compose, not override.** Each primitive enriches the model additively. No primitive needs to know what other primitives have done; the model accumulates state layer by layer.
3. **Primitives are typed model transforms.** Each primitive produces a specific kind of enrichment: new graph structure (overlay nodes and edges), allocated identifiers (IP addresses, ASNs, VLANs), compiled policy objects (firewall rules, route-maps), or protocol stanzas (VXLAN VNIs, EVPN route-targets). Primitives never name specific devices; they reference *groups* and *selectors* (graph queries that resolve against the current model state), so the same design plan can operate on topologies of different sizes.

2.6 Mini worked example (conceptual)

The following tiny design plan fragment illustrates the layered-model flow without requiring any knowledge of Rust, template engines, or selector internals.

Listing 1. A minimal design plan fragment (layered model + design rules).

```
layers:
- name: physical
  primitives:
  - type: mesh_nodes
    selector: "nodes[role=='core']"
    mesh_type: full

  - type: provision_ips
    selector: "edges[true]"
    pool: 10.0.0.0/24
    subnet_size: 30

- name: bgp
  requires: [physical]
  primitives:
  - type: build_protocol_layer
    selector: "nodes[layer=='physical' && role=='core']"
    layer: bgp
    clone_underlying: true
    config: { bgp: { enabled: true } }
```

```

# With clone_underlying: true, BGP overlay edges are derived
# automatically from the physical topology. No separate
# session primitive is needed.

assertions:
- name: core-has-link-address
  severity: error
  select: "nodes[role=='core']"
  check: { type: field_exists, field: src_ip }

```

Conceptually, this executes as follows:

1. **Start with an annotated topology** containing core routers as physical nodes.
2. **mesh_nodes** creates physical edges between those nodes.
3. **provision_ips** examines the edges to decide which links need subnets, then writes the resulting link addresses onto the endpoint *nodes* (not onto the edges).
4. **build_protocol_layer** clones the physical core nodes into a BGP overlay layer and records a parent mapping (*_source_id*) back to the physical devices. The protocol overlay config is deep-merged into the clones.
5. **Overlay edges encode protocol intent.** With `clone_underlying: true`, the BGP overlay begins with derived connectivity copied from the physical layer. The compiler then reads that overlay graph and writes per-device neighbour stanzas onto the participating nodes.
6. **Assertions** verify the model is well-formed (here: every core device has a link address) before any vendor-specific configuration is produced.

The key point is that the operator never names a specific interface or assigns an IP address directly. The design plan describes intent, and the compiler derives concrete per-device state from the layered topology relationships.

2.7 The select–enrich–validate pattern

Every primitive follows the same three-phase pattern (Figure 7):

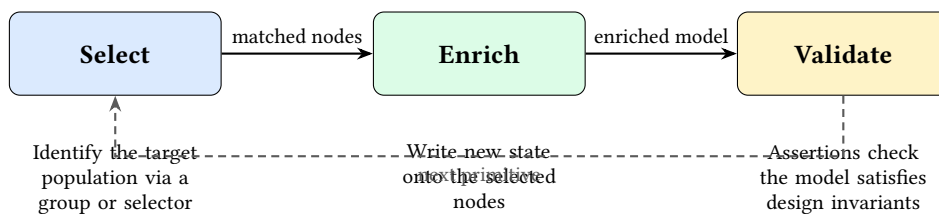


Figure 7. The select–enrich–validate cycle. Every primitive selects a population of nodes or edges and enriches them with new state. Validation is expressed as assertions over the model and may run as an explicit checkpoint, a layer barrier, or a final gate before rendering.

- **Select.** A group reference or `NETCFG` query identifies which nodes (or edges) the primitive will operate on. The selector is evaluated against the current graph state, so earlier primitives’ enrichments are visible to later selectors.
- **Enrich.** The primitive performs a typed model mutation on the selected population. Depending on the primitive’s tier, this may produce new graph structure (overlay nodes and edges), allocated identifiers (IP addresses, ASNs, VLAN IDs), compiled policy objects (firewall rules, route-maps, prefix-lists), or protocol stanzas (VXLAN VNIs, EVPN route-targets, BGP peering parameters). State always accumulates; primitives never delete or overwrite each other’s contributions.
- **Validate.** Assertions verify that the model satisfies the operator’s invariants (e.g., “every node has a link address” or “no two links share a subnet”). Assertions may run immediately (as checkpoints), at the end of a layer, and again as a final gate before any configuration is generated.

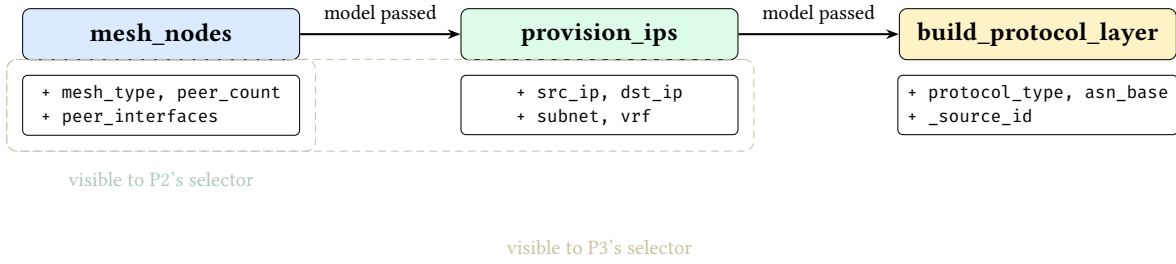


Figure 8. Selector visibility timeline. Each primitive’s selector evaluates against the *current* graph state, which includes all fields written by earlier primitives. State accumulates monotonically.

Validation checkpoints. NETCFG supports three complementary validation styles, all expressed as assertions over the current model state: (1) *mid-pipeline checkpoints* (via `assert_state`) that fail fast after a critical enrichment step; (2) *layer barriers* that run after a layer’s primitives execute; and (3) *final gates* that run before any configuration is rendered. Conceptually, all three ask the same question—“does the model satisfy the design invariants?”—but they fire at different times for faster feedback.

This uniform pattern means that learning one primitive teaches the operator the shape of every primitive: declare a selector, declare the desired state, and optionally assert invariants. The formal definitions and concrete DSL syntax follow in §3 and §II.

3 The Network Model

Concept first. The purpose of the formal model in this section is to make the compiler’s behaviour precise: what constitutes a node, what constitutes an edge, how layers coexist, and how per-device state accumulates. Implementation choices (Rust types, query engines, evaluation languages) are replaceable details; the stable ideas are: a layered graph, node-centric state, and deterministic derivation of device configuration from topology relationships.

NETCFG models the network as a directed graph $G = (V, E, \tau, \lambda, D)$ where V is a set of nodes (devices, protocol instances), $E \subseteq V \times V$ is a set of edges (links, peering sessions), $\tau : V \rightarrow T$ assigns each node a *type* (Router, Switch), $\lambda : V \rightarrow L$ assigns each node a *layer*, and $D : T \rightarrow \text{DataFrame}$ maps each type to a columnar property store (Polars DataFrame).

3.1 Layers

A layer represents a plane of the network: `physical` for the hardware topology, `bgp` for BGP peering sessions, `ospf` for OSPF adjacencies. A single physical router (node 1, layer `physical`) may have corresponding protocol nodes (node 101, layer `bgp`; node 201, layer `ospf`), each linked to node 1 via a *parent mapping* (`_source_id` in the node’s `data_json`).

Layers allow the compiler to model the physical and logical topologies simultaneously, with cross-layer references preserving traceability from a protocol node back to the physical device that hosts it.

3.2 Per-node state: DeviceContext

How per-node state relates to the layered model. The layered graph defines *where* intent lives: a device’s physical node captures hardware-adjacent facts (interfaces, link addressing, management), while each protocol-layer node captures logical intent for that protocol (BGP neighbours, OSPF areas, EVPN VNIs). In both cases, the compiler stores derived configuration-bearing state *on the node* so that rendering is a local operation: to generate configuration for a device in a given layer, the backend reads exactly one node’s accumulated state.

Conceptually: a node’s “accumulated facts”. Think of `DeviceContext` as a typed view over a node’s accumulated facts at that point in the pipeline. Primitives add facts (“this interface has 10.0.0.1/30”, “this device is in VRF X”, “these are my BGP peers”), and assertions check that required facts are present before lowering continues.

Each node’s properties are stored as a JSON blob in a Polars DataFrame keyed by node type. Polars provides ultra-fast topological graph queries and edge traversals, unpacking the JSON payload on-demand only when evaluating NetCfg Query Language (NQL) expressions (which compile to CEL [12] internally). The compiler maintains a per-node *device context* whose fields are progressively enriched by primitives:

Table 3. Per-node device context: fields populated by the compiler pipeline. Fields marked with † are written by specific primitives; all others originate from the annotated topology.

Field	Type	Source
hostname	string	Topology node id
device_os	string?	Topology annotation
management_ip	string?	Topology annotation
src_ip, dst_ip, subnet	string?	Written by provision_ips †
src_ipv6, dst_ipv6, subnet_v6	string?	Written by provision_ips † (dual-stack)
vrf	string?	Topology annotation or allocate_resources †
peer_interfaces	map	Written by mesh_nodes †
stanzas	list	Accumulated config units from all primitives †
(any other key)	JSON value	Topology annotations or primitive-written metadata

Primitives progressively enrich this context: `mesh_nodes` establishes edges and writes `mesh_type/peer_count` into metadata; `provision_ips` writes IP fields; `build_protocol_layer` deep-merges config overlays into the cloned node’s metadata. By the time rendering occurs, each node’s device context contains all information needed to produce its configuration file.

Note

Any topology annotation that does not match a named field is captured in the node’s metadata map. This makes the device context extensible without schema changes—operators can attach arbitrary properties in the topology and reference them from selectors or templates.

Note

Graph engine dependency. The NTE topology engine [13] (NTE-TR-001) provides the directed graph with columnar (Polars DataFrame) property storage. NETCFG uses NTE for the graph, the mapping evaluator, and DeviceIR types. NETCFG’s `mesh_nodes` and `build_protocol_layer` primitives rely on the Endpoints Model—a 3-hop traversal pattern (NTE-TR-001 §Data Model) that discovers adjacent devices via the DWC (Device–Wire–Cable) abstraction.

4 Intent and Policy Expression

The operator formalises a high-level architectural plan into two core artefacts: an annotated topology (the “source code” of the network) and a declarative design plan. The plan dictates how the topology should be projected into distinct **Network Layers** (e.g., physical, BGP, OSPF). The compiler synthesises these layers, resolving cross-layer dependencies, before finally mapping the enriched state down to the hardware-specific **DeviceIR**. The operator never manipulates IP addresses or protocol stanzas directly; the compiler derives them through this layer-by-layer enrichment process. This directly maps architectural intent to formal logic.

This section frames the conceptual model that operators use when authoring these artefacts, before §II catalogues the concrete syntax.

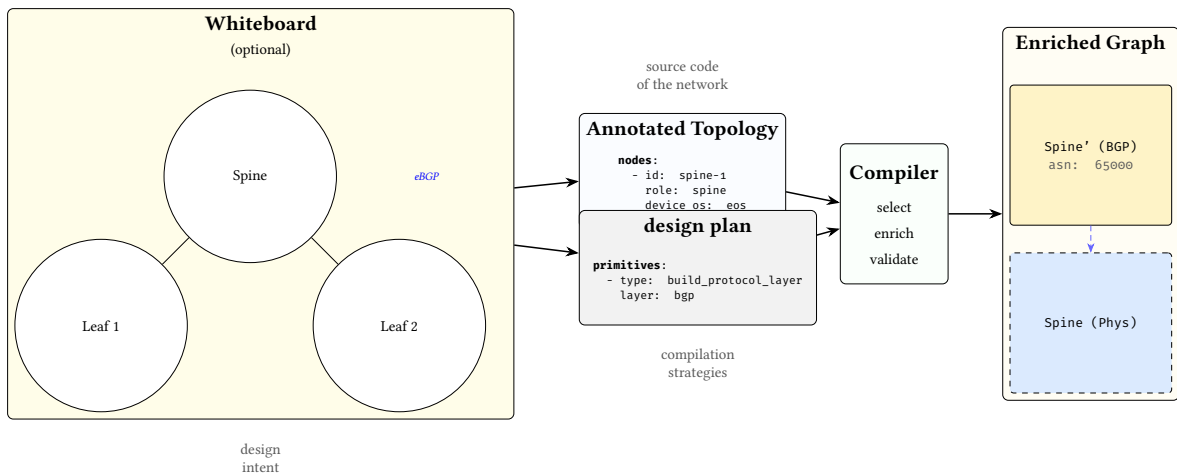


Figure 9. From whiteboard to compiled model. An architect’s visual intent (left, optional) informs both the annotated topology (the network’s “source code”: nodes, roles, sites, properties) and the design plan (which compilation strategies to apply). Together, these feed the compiler, which produces an enriched, layered graph (right).

4.1 The five concerns of a network design plan

Every design plan addresses five orthogonal concerns. Table 4 shows how each maps to a DSL construct and the design question it answers.

Table 4. The five concerns of a network design plan.

Concern	DSL construct	Question answered
Population naming	groups:	Which devices belong to which roles?
Topology intent	layers: / primitives	How should they be connected and what state should they carry?
Policy constraints	routing/access/prefix-list primitives	What traffic rules should be enforced?
Design invariants	assertions: + rules:	What properties must always hold?
Change safety	blast_radius:	How much change is acceptable per commit?

The key insight is that every concern operates on the *network model* —the layered graph described in §3. The operator declares populations, intent, policy, and invariants; the compiler maps these into graph mutations that progressively enrich the model’s nodes.

4.2 From plan to network model: declarative composition

§2.5 introduced the five primitive tiers and their mutation semantics. This subsection shows how those tiers interact at the implementation level.

`provision_ips` iterates edges to determine which node pairs need addresses, but writes `src_ip` and `dst_ip` onto the respective *nodes*—not onto the edge itself. `build_protocol_layer` clones selected nodes into a new graph layer and merges protocol configuration into them via deep merge. Each primitive sees the enriched model left by its predecessors, but never needs to know what they did.

: Node-Centric State

All per-device state is written to nodes via `DeviceContext`. Edges carry no mutable state; they exist solely to express relationships between nodes. This invariant simplifies the mapping stage: to produce a device’s configuration, the mapping evaluator reads exactly one node.

Groups and selectors are the glue that lets these transforms find their targets. Groups let the operator name populations once (e.g. `$core_routers`, `$edge_links`, `$management_nodes`) and reference them in every primitive and assertion. Selectors compile group references and attribute predicates into NQL graph queries that the compiler evaluates against the live topology.

4.3 Expressing policy at the plan layer

Policy primitives follow the same declarative pattern: select a population, declare the desired policy state, and let the compiler write it to the model. The operator never constructs individual ACL entries or route-map clauses. Instead, they declare the *intent*—which traffic classes to permit or deny, which prefixes to advertise, which communities to match—and the primitive translates this into the appropriate policy objects on each selected node.

The separation between *topology intent* (layers 1–2: structure and addressing) and *policy intent* (layer 3+: traffic rules) is a deliberate design choice. Topology primitives build the graph; policy primitives annotate nodes with forwarding constraints. Both operate through the same select-and-enrich mechanism, but they address orthogonal concerns and can be developed, tested, and imported independently.

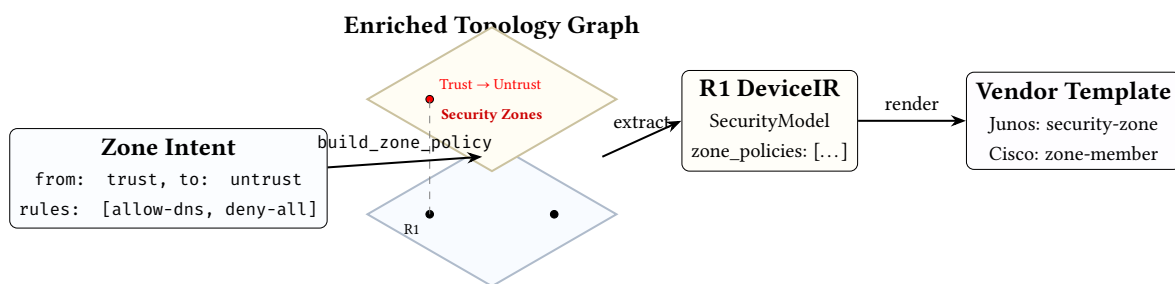


Figure 10. Policy compilation flow. The operator declares declarative zone intent; the `build_zone_policy` primitive synthesises this into a logical security overlay within the topology graph. This state is then extracted into DeviceIR rule objects and finally rendered via vendor templates.

Case study: multi-tenant isolation

The multi-tenant data-centre case study (`examples/case_studies/04_multi_tenant_dc.yaml`) illustrates how these concerns compose. The operator declares: (1) groups that name device roles and per-tenant link populations; (2) a hub-and-spoke mesh for the fabric; (3) per-VRF address pools; (4) access policies that deny cross-VRF traffic; (5) assertions that every device has a tenant assignment and that each tenant’s subgraph is connected; and (6) a blast-radius limit. At no point does the operator name a specific device or IP address—the compiler derives all concrete state from the declared populations and intent.

4.4 Reusable design-rule libraries

Complex assertions can be factored into named `rules:` (NQL macros) and shared across design plans via `imports:`. Two standard rule libraries and a protocol fragment library ship with `NETCFG`:

- **datacenter-rules.yaml** — a portable linting ruleset for data-centre fabrics. It validates OSPF point-to-point subnet lengths, MTU consistency across edges, BGP private-ASN ranges, BGP neighbour IP existence, IS-IS level consistency, and IS-IS NET address formatting. design plans import it with a single line: `imports: [ 'datacenter-rules.yaml' ]`.
- **hardware-lowering.yaml** — vendor-specific AST transforms for Cisco NX-OS, Juniper Junos, Cisco IOS-XR, and VyOS. It remaps abstract Ethernet interface names into platform-specific formats (e.g. `Ethernet0` → `ge-0/0/0` on Junos) and standardises overlay MTU to 9216.
- **protocols/*.yaml** — a library of 31 importable protocol fragments, each providing a single `build_protocol_layer` invocation with sensible defaults. Covers routing (BGP, OSPF v2/v3, IS-IS, RIP, LDP, RSVP-TE, SR-MPLS), switching (VXLAN, EVPN, STP/RSTP, LACP, MLAG), se-

curity (IPsec/IKEv2, WireGuard, MACsec, 802.1X, RADIUS, TACACS+), services (NTP, Syslog, SNMP, DHCP Relay), multicast (PIM-SM, IGMP), telemetry (NetFlow/IPFIX/sFlow), and layer 2 (LLDP, ARP, BFD, VRRP, GRE). design plans import individual protocols: imports: ['protocols/ospf.yaml', 'protocols/bfd.yaml'].

Ten case studies in examples/case_studies/ demonstrate production-scale design plans: ISP dual-stack peering, campus compliance audit, WAN OSPF→IS-IS migration, multi-tenant DC with per-VRF isolation, SP MPLS core with SR-MPLS, campus 802.1X NAC, SD-WAN with IPsec overlay, multicast video distribution, zero-trust DC microsegmentation, and PCI-DSS financial compliance.

Assertions reference named rules via the use_rule check type, keeping the assertion block concise:

Listing 2. Referencing named rules via use_rule.

```
rules:
  link-addressed: "has(node.src_ip) && node.src_ip != ''"
  protocol-bound: "has(node.metadata.asn) && node.metadata.asn != ''"

assertions:
- name: every-edge-device-has-address
  severity: error
  select: nodes[role=='edge']
  check:
    type: use_rule
    name: link-addressed
```

5 The Compilation Pipeline

This section describes the pipeline conceptually; Part III (§16) covers each layer’s implementation.

NETCFG’s compilation pipeline has five distinct layers of abstraction, operating as a true network compiler. Figure 11 shows the data flow from abstract intent to concrete syntax.

A key property of this pipeline is **determinism**: configuration is a pure function $f(\text{Annotated Topology, design plan}) = \text{DeviceIR} \rightarrow \text{Config}$. If neither the topology annotations nor the design plan strategies have changed, regenerating the configuration produces a bit-identical result, making any divergence immediately detectable via diff. Combined with the structural diff engine (§17.3), this enables a “regenerate and compare” workflow for detecting configuration drift without runtime monitoring.

Note

The conference paper describes six implementation stages (parse, shadow, execute, map, render, write); this report groups these into five conceptual layers (Intent, Assertions, Mapping/DeviceIR, TSDM, Syntax). Both describe the same pipeline at different levels of abstraction: the conference paper’s “parse” and “shadow” stages are subsumed by Layer 1 (Intent Transformation), and “render” and “write” are subsumed by Layer 5 (Syntax Serialisation).

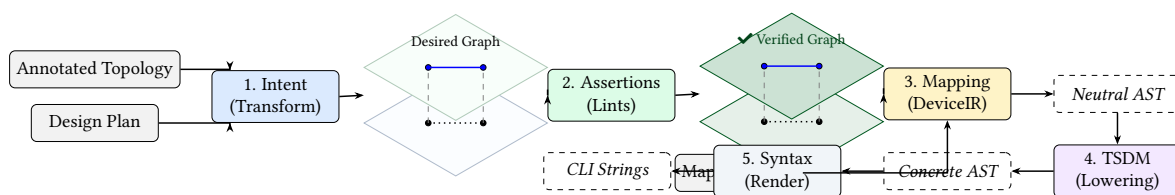


Figure 11. NetCfg compilation pipeline. High-level intent is progressively lowered through three distinct AST representations (Verified Graph, Neutral AST, Concrete AST) before serialisation.

6 Related Work

Network configuration synthesis and verification have received significant attention. We position NETCFG against three strands of related work.

Configuration synthesis and formal methods. Early declarative networking languages like Frenetic [14] and NetKAT [15] pioneered the separation of intent from mechanism via formal semantics in SDN, but did not address legacy CLI-driven environments. Propane [16] compiles high-level routing policies written in a purpose-built language into per-device BGP configurations, proving correctness by construction. SyNET [17] uses constraint solving to synthesise router configurations from a formal network specification. Both systems target routing exclusively; NETCFG covers a broader surface (IP allocation, zone policies, NAT, overlays) at the cost of weaker formal guarantees—design-rule assertions rather than proofs. On the data plane, P4 [18] demonstrated that packet-processing hardware could be programmed via a compiler funnel that lowers abstract intent into target-specific architectures. NETCFG applies this exact compiler philosophy to the management plane.

Configuration analysis and verification. Header Space Analysis [19] introduced the foundation for statically checking network properties. Building on this, Batfish [20] models device configurations as a data plane and evaluates reachability queries offline; Minesweeper [21] extends this with symbolic model checking against network-wide invariants. Config2Spec [22] mines specifications from existing configurations to bootstrap verification. Forward Enterprise [23] provides commercial intent verification with a digital-twin approach. These tools analyse *existing* configurations, whereas NETCFG operates upstream: it *generates* the configurations that such tools would later verify, and its assertion framework provides lightweight invariant checking at compile time.

Operational tooling and orchestration. Ansible [8], Salt [24], and Nornir [9] are widely deployed for configuration deployment and orchestration. Cisco NSO [25] provides model-driven (YANG-based) service orchestration. netlab [26] generates lab topologies for testing. These tools focus on *deploying* or *simulating* configurations rather than synthesising them from intent. Frenetic [14] introduced a functional network programming language for SDN controllers, inspiring the declarative philosophy that NETCFG applies to traditional device configuration.

NETCFG occupies a middle ground: it is more expressive than pure routing synthesisers, less formally rigorous than model checkers, and operates at a higher abstraction level than deployment tools. Its graph-based compilation pipeline (§5) and pluggable primitive architecture (§11) are, to our knowledge, unique in this space.

THE DESIGN PLAN LANGUAGE

The following sections specify the design plan language—the declarative surface that expresses the “what” of a network design. Together they constitute the complete DSL reference.

7 Structuring the compilation plan

The plan organizes design rules into ordered compilation layers. Each layer executes sequentially, allowing logical topologies to be built on top of physical structures.

Listing 3. Design Plan top-level schema.

```
version: 1                                # schema version (required)
imports: [shared-rules.yaml]              # merge other design plans
layers:                                   # ordered compilation layers
  - name: physical
    requires: []                           # layer dependencies
    primitives: [...]                     # typed model transforms
assertions: [...]                        # post-execution design checks
rules:                                   # named NQL macros
  is_core: "role == 'core'"
groups:                                   # named node sets
  core_routers: { selector: "nodes[role=='core']" }
```

```

blast_radius: [...]           # change safety policies
transforms: [...]             # vendor-specific adaptation
hardware_library: { ... }     # chassis/linecard definitions

```

Three validation passes run at parse time:

1. **Schema validation:** field types, non-empty names, integer ranges. Unknown schema keys are rejected at parse time, catching typos before execution.
2. **NETCFG query compilation:** each selector string is compiled into an evaluable query program, surfacing syntax errors before execution.
3. **Layer ordering validation:** a forward scan ensures every requires entry names a previously-declared layer. This catches ordering bugs at parse time.

The optional `imports` field lists relative file paths to other design plan files. At parse time, layers and assertions from imported files are merged into the importing design plan, enabling reusable assertion libraries (e.g. a standard linting ruleset shared across projects).

Warning

Duplicate layer names are permitted. Multiple `LayerSpec` blocks with `name: input` are common in practice—each block adds primitives to the same layer. The `requires` mechanism enforces inter-layer ordering, not intra-layer uniqueness.

8 Selector DSL (NQL Surface Syntax)

The NETCFG Query Language (NQL)

NQL is the graph-traversal language that drives NETCFG. Traditional automation uses explicit `for` loops and `if` statements in Python or Jinja; NQL replaces these with mathematical predicates over nodes and edges (e.g., “all spine routers in London”). The compiler translates each NQL selector into an optimised graph traversal against the NTE engine. NQL compiles down to NTE-QL (NTE-TR-001 \$Query System).

Selectors identify which nodes or edges a primitive operates on. Conceptually, a selector is simply a boolean query over the current model: “for each node (or edge), should this primitive apply here?” The concrete surface syntax is a domain-specific dialect compiled down to Google’s Common Expression Language (CEL) [12] under the hood:

Listing 4. Selector syntax examples.

```

# Select all nodes
selector: "all()"

# Select nodes by property (e.g. region, role, type)
selector: "nodes[region=='eu-west']"
selector: "nodes[role=='core' && layer=='input']"
selector: "nodes[type=='Router' and region=='us-east']"

# Select edges
selector: "edges[src>=0]"
selector: "edges[true]"

```

The selector parser:

1. Identifies whether the selector targets nodes or edges and extracts the bracketed predicate.
2. Normalises minor syntax differences (e.g. `and` vs `&&`) so the predicate is unambiguous.
3. Compiles the predicate into an evaluable program.

At evaluation time, the selector ranges over all nodes (or edges) in the topology and returns the subset for which the predicate is true. This means selectors are always evaluated against the *current* model state: earlier primitives can add fields that later selectors use.

Mechanically, the compiler binds each entity’s properties as variables (id, type, layer, plus all data_json fields, flattened and accessible via dot notation) and evaluates the compiled predicate.

Note

Why missing fields are not errors. Because the model is layered, not every node has every field: a physical node will have interface and addressing facts, while a protocol node will have protocol-specific facts. Selectors must therefore tolerate heterogeneous populations. Mechanically, undeclared variable references (e.g., querying a field that some nodes lack) silently evaluate to false rather than producing an error. This lets a single selector range over mixed layers while still precisely matching only the nodes that carry the relevant facts.

For IP-valued fields, the selector also binds a {field}_len variable containing the prefix length, enabling selectors like nodes[subnet_len == 30].

Note

Layer-aware selector patterns. Because the model is layered, selectors typically constrain both *what* a node is (role/type) and *where* it lives (layer). Common patterns include:

```
# Physical devices only
selector: "nodes[layer=='physical' && role=='core']"

# Protocol overlay nodes only (e.g. BGP layer)
selector: "nodes[layer=='bgp' && role=='core']"

# Edges within a layer (physical links, protocol sessions)
selector: "edges[layer=='physical']"

# Guard against heterogeneous fields (match only nodes that have the facts)
selector: "nodes[layer=='bgp' && has(node.bgp) && node.bgp.enabled==true]"
```

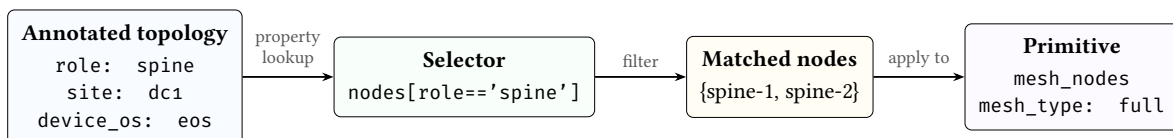


Figure 12. Selector-to-annotation data flow. Topology annotations are the values that selectors query; the selector filters nodes into a matched set; the primitive then operates on that set. The topology annotations are the “input language” and the selector is the bridge between topology intent and design plan strategy.

9 Named groups

The groups: section defines reusable named node or edge sets. Group names are prefixed with \$ when referenced in selectors. Two forms are supported:

Listing 5. Group definitions: shorthand and full form.

```
groups:
  # Shorthand: just a selector string (e.g. by role)
  core_routers: "nodes[role=='core']"
  access_switches: "nodes[role=='access']"

  # Full form with kind and kind-specific fields
  dmz:
    selector: "nodes[zone=='dmz']"
    kind: security_zone
    trust_level: 10
```

Groups are resolved before primitives execute. The selector parser expands \$group_name references inline, and cycle detection rejects circular group dependencies at parse time. When design plans

import other files, groups from imported files are merged into the importing design plan's namespace (name collisions are last-writer-wins).

The union operator (`|`) combines groups:

```
all_devices: $core_routers | $access_switches
```

This creates a set containing all nodes matching either group.

When a group has `kind: security_zone`, an optional `interface_selector` field may be supplied to express interface-level zone membership. This selector targets edges rather than nodes, enabling fine-grained policies where different interfaces on the same device belong to different zones:

```
dmz:
  selector: "nodes[zone=='dmz']"
  kind: security_zone
  trust_level: 10
  interface_selector: "edges[intf_role=='dmz']"
```

10 Named objects

The top-level `objects:` section defines reusable address and service objects. These are referenced in zone policy and NAT policy rules using the `$` prefix (e.g. `$rfc1918`, `$web_services`). Both sub-maps default to empty, so omitting `objects:` is backward-compatible.

Listing 6. Named address and service objects.

```
objects:
  addresses:
    rfc1918:
      prefixes:
        - "10.0.0.0/8"
        - "172.16.0.0/12"
        - "192.168.0.0/16"
    corp_v6:
      prefixes_v6:
        - "2001:db8::/32"
  services:
    web_services:
      entries:
        - { protocol: tcp, port: 80 }
        - { protocol: tcp, port: 443 }
    dns:
      entries:
        - { protocol: udp, port: 53 }
        - { protocol: tcp, port: 53 }
```

`AddressObject` supports dual-stack addressing via separate `prefixes` (IPv4) and `prefixes_v6` (IPv6) fields. Prefixes are stored as strings and validated to `ipnet::IpNet` at object registry build time.

`ServiceObject` contains one or more `ServiceEntry` pairs of `protocol` and `port`. In zone policy or NAT rules, use `service: $web_services` in the `match:` block to reference a named service object.

11 Primitive catalogue

A *primitive* is the atomic unit of compilation: a declarative instruction that reads from the topology model, computes derived state, and writes enrichments back. Primitives compose within layers (declared execution order) and across layers (dependency ordering). The five tiers below reflect what each primitive mutates: topology structure, per-node state, cross-device relationships, policy constraints, or meta-operations on the pipeline itself.

`NETCFG` provides twenty-three primitive types organised into five tiers, each distinguished by what it *mutates* in the network model.²

²The `wasm_plugin` primitive requires the optional `wasm Cargo` feature; a default build exposes twenty-two primitives.

Note: primitive naming

The primitive types documented below use their stable DSL aliases (e.g. `allocate_resources`, `build_routing_policy`). The compiler internally uses canonical names (e.g. `provision_resources`, `policy_routing`); the DSL aliases are accepted for backward compatibility. Both names are valid in design plan YAML; the canonical name appears in error messages and diagnostic output. Table 5 lists all renames.

Table 5. Primitive canonical names and DSL aliases.

Canonical (Rust)	DSL alias	Tier
<code>provision_resources</code>	<code>allocate_resources</code>	Allocation
<code>define_ip_pool</code>	<code>global_pool</code>	Allocation
<code>define_resource_pool</code>	<code>global_resource_pool</code>	Allocation
<code>policy_routing</code>	<code>build_routing_policy</code>	Policy
<code>policy_access</code>	<code>build_access_policy</code>	Policy
<code>policy_prefix_list</code>	<code>build_prefix_list</code>	Policy
<code>policy_community_list</code>	<code>build_community_list</code>	Policy
<code>policy_bgp_safety</code>	<code>generate_safe_bgp_filters</code>	Policy
<code>overlay_vxlan</code>	<code>build_vxlan</code>	Overlay
<code>overlay_evpn</code>	<code>build_evpn</code>	Overlay

The five tiers are:

- **Topology** — create new nodes and edges (graph structure).
- **Allocation** — assign derived identifiers to existing nodes (IP addresses, ASNs, VLANs).
- **Policy** — compile high-level declarations into per-device rule sets (zone policies, ACLs, route-maps).
- **Overlay** — attach protocol-specific configuration stanzas (VXLAN, EVPN, BGP peering).
- **Meta** — annotate, gate, or extend the pipeline without producing device configuration.

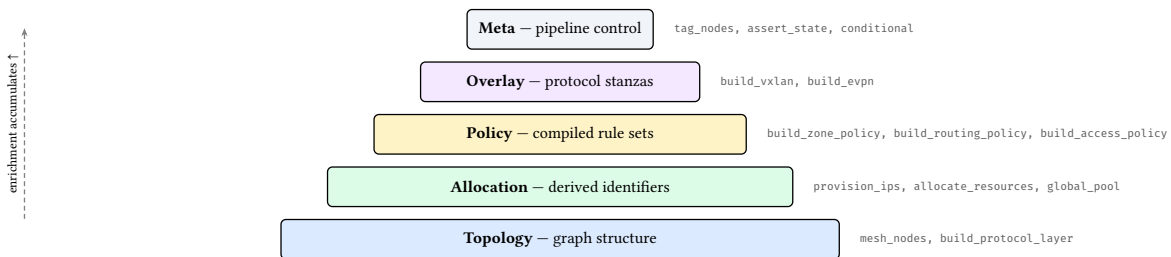


Figure 13. Primitive tier hierarchy. Lower tiers create structure; upper tiers annotate and configure existing nodes. Each tier builds on state written by earlier tiers.

All primitives follow the *select* → *enrich* → *validate* pattern: select a target population via graph queries, compute the enrichment, then write results to DeviceContext or the graph structure.

Tables 6–8 summarise the catalogue. Full field tables follow.

Table 6. Design rules for Topology & Protocol Overlays. Note that specific routing protocols (BGP, OSPF, IS-IS) are not separate primitives; they are all instantiated via the generic `build_protocol_layer` rule.

Design Rule	Category	Purpose
<code>mesh_nodes</code>	Topology	Create physical or logical edges between selected nodes (full, hub-and-spoke)
<code>build_bgp</code>	Protocol	Project nodes into a BGP overlay (allocates ASNs, establishes peering)
<code>build_ospf</code>	Protocol	Project nodes into an OSPF area (computes network types and adjacencies)
<code>build_isis</code>	Protocol	Project nodes into an IS-IS level (derives NET addresses and adjacencies)
<code>build_bfd</code>	Protocol	Attach BFD link-failure detection to underlying routing adjacencies
<code>build_multicast</code>	Protocol	Project nodes into a PIM/IGMP multicast distribution tree

Table 7. Design rules for Policy & Resource Allocation.

Design Rule	Category	Purpose
<code>provision_ips</code>	Allocation	Allocate IP subnets to edges from a CIDR pool
<code>allocate_resources</code>	Allocation	Generic pool allocation for ASNs, VLANs, etc.
<code>global_pool</code>	Allocation	Register a named IP pool for hierarchical carving
<code>global_resource_pool</code>	Allocation	Register a named resource pool (non-IP)
<code>build_routing_policy</code>	Policy	Attach routing policy stanzas (route-maps)
<code>build_access_policy</code>	Policy	Attach access-control policy stanzas (ACLs)
<code>build_prefix_list</code>	Policy	Create named prefix-list entries on matched nodes
<code>build_community_list</code>	Policy	Create named community-list entries on matched nodes
<code>generate_safe_bgp_filters</code>	Policy	Auto-generate RFC 1918 bogon filters
<code>build_zone_policy</code>	Policy	Zone-based firewall policies with ordered rules
<code>build_nat_policy</code>	Policy	NAT policies (SNAT interface/pool, DNAT port-forward)
<code>build_qos_policy</code>	Policy	QoS policies (classifier, scheduler, policer, shaper)

Table 8. Design rules for Overlays & Metadata.

Design Rule	Category	Purpose
<code>build_vxlan</code>	Overlay	Assign VXLAN VNI and multicast group bindings
<code>build_evpn</code>	Overlay	Assign EVPN route-distinguisher and route-target
<code>tag_nodes</code>	Meta	Stamp key-value metadata onto matched nodes
<code>map_hardware_inventory</code>	Meta	Assign chassis model and line-card slot mappings
<code>assert_state</code>	Meta	Mid-pipeline assertion checkpoint
<code>conditional</code>	Meta	Query-based branching within design plans
<code>custom_primitive</code>	Meta	Named reusable design-rule group with parameters
<code>inject_secrets</code>	Meta	Read environment variables into node metadata
<code>wasm_plugin</code>	Meta	Execute a WebAssembly plugin against matched nodes

11.1 mesh_nodes

Creates edges between selected nodes. Supports two mesh topologies: full ($\binom{n}{2}$ edges) and hub_and_spoke (hubs full-meshed, spokes connected to all hubs). Idempotent: existing edges are detected via a `HashSet<(i32, i32)>` and skipped.

Table 9. mesh_nodes fields.

Field	Type	Required	Description
selector	String	Yes	NETCFG query for nodes to mesh
mesh_type	String	Yes	full or hub_and_spoke
hub_selector	String	H&S	Selector for hub nodes (hub-and-spoke only)
spoke_selector	String	H&S	Selector for spoke nodes (hub-and-spoke only)

Example: full mesh within a site

```
- type: mesh_nodes
  selector: "nodes[site='a']"
  mesh_type: full
```

With 4 nodes at site A, this creates $\binom{4}{2} = 6$ edges and assigns interface names eth0, eth1, etc. to each node's peer_interfaces map.

11.2 provision_ips

Allocates subnets from a CIDR pool to edges. See §17.1 for the full five-pass algorithm.

Table 10. provision_ips fields.

Field	Type	Required	Description
selector	String	Yes	NETCFG query for edges
pool	String	Yes	CIDR (e.g. 10.0.0.0/24) or global pool name
subnet_size	u8	No	Prefix length per edge (default: 30 for IPv4, 126 for IPv6)
ipv6_pool	String	No	IPv6 CIDR or global pool name for dual-stack
ipv6_subnet_size	u8	No	IPv6 prefix length per edge
strategy	String	No	dense (default), sparse, or hash
pool_size	u8	No	When referencing a global pool, prefix length of block to carve
vrf	String	No	VRF domain (default: default)

11.3 build_protocol_layer

Clones physical nodes into a named protocol layer. See §17.2 for the overlay construction mechanism.

Table 11. build_protocol_layer fields.

Field	Type	Required	Description
selector	String	Yes	NETCFG query for source-layer nodes
layer	String	Yes	Target layer name (must differ from source)
config	JSON	Yes	Config overlay deep-merged into cloned nodes
clone_underlying	bool	No	Also clone edges between selected nodes (default: false)

11.4 allocate_resources

Generic pool allocator for non-IP resources (ASNs, VLANs, loopback IDs).

Table 12. allocate_resources fields.

Field	Type	Required	Description
selector	String	Yes	NETCFG query for nodes
resource_type	String	Yes	Key name in DeviceContext.metadata
pool	String	Yes	Range (e.g. 65000-65999) or global pool name
strategy	String	No	dense (default), sparse, or hash
pool_size	u32	No	When referencing a global pool, number of values to carve

11.5 global_pool

Registers a named IP address pool at design plan scope, enabling hierarchical pool carving across multiple provision_ips primitives.

Table 13. global_pool fields.

Field	Type	Required	Description
name	String	Yes	Pool identifier referenced by provision_ips
pool	String	Yes	CIDR range (e.g. 10.0.0.0/8)
vrf	String	No	VRF domain (default: default)

11.6 global_resource_pool

Registers a named non-IP resource pool at design plan scope.

Table 14. global_resource_pool fields.

Field	Type	Required	Description
name	String	Yes	Pool identifier
resource_type	String	Yes	Resource category (e.g. asn)
pool	String	Yes	Value range (e.g. 65000-65999)

11.7 conditional

Query-based branching. The condition is evaluated as a NETCFG query expression; if true, then_primitives execute, otherwise else_primitives (if present) execute.

Table 15. conditional fields.

Field	Type	Required	Description
condition	String	Yes	NETCFG query expression
then_primitives	Vec<Primitive>	Yes	Primitives if condition is true
else_primitives	Vec<Primitive>	No	Primitives if condition is false

11.8 custom_primitive

A named, reusable group of primitives with parameters. The parameters map is pushed onto a stack and accessible within the nested primitives.

Table 16. custom_primitive fields.

Field	Type	Required	Description
name	String	Yes	Primitive name (for reporting)
parameters	map (string → value)	Yes	Named parameters
primitives	list of primitives	Yes	Nested primitive list

11.9 inject_secrets

Reads environment variables into node metadata. Keys in the `secrets` map are metadata field names; values are environment variable names.

Table 17. inject_secrets fields.

Field	Type	Required	Description
selector	String	Yes	NETCFG query for target nodes
secrets	map (string → string)	Yes	{metadata_key: env_var_name}

11.10 build_routing_policy

Attaches routing policy stanzas to matched nodes. Each statement defines a route-map entry with match conditions and set actions, stored as a `Stanza` in `DeviceContext.stanzas` for downstream template rendering.

Table 18. build_routing_policy fields.

Field	Type	Required	Description
selector	String	Yes	NETCFG query for target nodes
policy_name	String	Yes	Policy name (e.g. BGP-EXPORT)
statements	list of statements	Yes	Ordered route-map entries

Each `RoutingPolicyStatement` has fields: `name` (sequence label), `action` (permit or deny), and optional match/set clauses: `match_prefix_list`, `match_community_list`, `match_as_path`, `set_local_preference` (integer), `set_metric` (integer), `set_as_path_prepend`, `set_community`, and `next_hop` (IP address).

Example: BGP export policy

```
- type: build_routing_policy
  selector: "nodes[role='border']"
  policy_name: "BGP-EXPORT"
  statements:
    - name: "10"
      action: permit
      match_prefix_list: "CUSTOMER"
      set_local_preference: 100
    - name: "20"
      action: deny
```

11.11 build_access_policy

Attaches access-control policy stanzas to matched nodes. Each rule defines a firewall or ACL entry with source, destination, protocol, and port fields.

Table 19. build_access_policy fields.

Field	Type	Required	Description
selector	String	Yes	NETCFG query for target nodes
policy_name	String	Yes	Policy name (e.g. EDGE-ACL)
rules	list of rules	Yes	Ordered ACL entries

Each AccessPolicyRule has fields: name, action (permit/deny), optional source_prefix (CIDR), destination_prefix (CIDR), protocol (string), source_port (integer), and destination_port (integer). Both policy primitives enforce node targeting—policies cannot target edges.

Example: DMZ access control list

```
- type: build_access_policy
  selector: "nodes[zone='dmz']"
  policy_name: "UNTRUSTED-TO-INTERNAL"
  rules:
    - name: "100"
      action: deny
      protocol: tcp
      destination: "10.0.0.0/8"
    - name: "999"
      action: permit
```

11.12 wasm_plugin

[Experimental] Executes a WebAssembly plugin binary against matched nodes. Requires the wasm Cargo feature flag (`-features wasm`); without it, the pipeline returns a descriptive error. Uses the wasmtime [27] v18 runtime.

Table 20. wasm_plugin fields.

Field	Type	Required	Description
selector	String	Yes	NETCFG query for target nodes
plugin_path	String	Yes	Filesystem path to compiled .wasm binary

The execution pipeline: (1) load and compile the .wasm module, (2) instantiate per matched node without host imports, (3) resolve the apply_node exported symbol, (4) pass the node's DeviceContext as serialised JSON. The current implementation validates compilation and symbol resolution; full linear memory I/O for JSON exchange is planned for a future release (see §24).

Note

The wasm feature is optional to avoid the wasmtime dependency (which adds ~40 MB to the binary) for users who do not need plugin extensibility.

11.13 build_prefix_list

Creates named prefix-list entries on matched nodes, stored as Stanza objects for downstream template rendering.

Table 21. build_prefix_list fields.

Field	Type	Required	Description
selector	String	Yes	NETCFG query for target nodes
prefix_list_name	String	Yes	Name of the prefix list
entries	Vec<PrefixListEntry>	Yes	Ordered prefix-list entries

Each PrefixListEntry has: prefix (CIDR string), action (permit/deny), optional le (less-than-or-equal prefix length), and optional ge (greater-than-or-equal prefix length).

11.14 build_community_list

Creates named community-list entries on matched nodes.

Table 22. build_community_list fields.

Field	Type	Required	Description
selector	String	Yes	NETCFG query for target nodes
community_list_name	String	Yes	Name of the community list
entries	Vec<CommunityListEntry>	Yes	Ordered community-list entries

Each CommunityListEntry has: community (e.g. 65001:1000) and action (permit/deny).

11.15 generate_safe_bgp_filters

Auto-generates RFC 1918 bogon filters as prefix-list and routing-policy stanzas on matched nodes.

Table 23. generate_safe_bgp_filters fields.

Field	Type	Required	Description
selector	String	Yes	NETCFG query for target nodes
prefix_list_name	String	No	Name for the generated prefix list
policy_name	String	No	Name for the generated routing policy
block_rfc1918	bool	No	Block RFC 1918 prefixes (default: true)
permit_default	bool	No	Add a trailing permit-any (default: false)

11.16 build_zone_policy

Compiles a zone-based firewall policy for traffic flowing between two security zones. The operator declares the zone pair, an ordered list of match/action rules, and a default action (deny by default). At execution time, the primitive validates zone names against the group registry, constructs a typed SecurityModel on each matched node, and appends the zone policy to the node's DeviceContext. Vendor templates render the policy into platform-specific syntax (Junos security-zone ACLs, Cisco ZBFW class-maps, VyOS zone-policy rules, etc.).

Table 24. build_zone_policy fields.

Field	Type	Required	Description
selector	String	Yes	NETCFG query for target devices
from_zone	String	Yes	Source security zone name
to_zone	String	Yes	Destination security zone name
default_action	String	No	permit or deny (default: deny)
rules	Vec<Rule>	Yes	Ordered list of match/action rules

Each rule has a name, an action (permit/deny), and an optional match block with fields: source, destination, service, protocol, destination_port, icmp_type. Optional log and stateful flags control logging and connection-tracking behaviour.

11.17 build_nat_policy

Attaches NAT policy rules to matched devices. Supports four rule types: snat (source NAT via interface or pool), dnat (destination NAT / port-forwarding), nat64 (RFC 6146 stateful IPv6-to-IPv4 translation), and nptv6 (RFC 6296 IPv6 prefix translation). Rules are ordered and may optionally reference zone pairs (from_zone, to_zone) for zone-scoped NAT.

Table 25. build_nat_policy fields.

Field	Type	Required	Description
selector	String	Yes	NETCFG query for target devices
rules	Vec<NatRule>	Yes	Ordered list of NAT rules

Each NatRule has a name, a type (snat/dnat/nat64/nptv6), an optional match block, and a type-specific parameter block. SNAT rules specify mode (interface or pool) and optional pool_addresses. DNAT rules specify external_address, optional external_port, internal_address, and optional internal_port. NAT64 rules specify a nat64_prefix; NPTv6 rules specify internal_prefix, external_prefix, and an optional allow_best_effort flag. Rules may also carry from_zone/to_zone for zone-scoped NAT and an optional log flag.

The match block accepts four optional fields:

Table 26. NAT rule match fields (NatMatchSpec).

Field	Type	Description
source	String	Source CIDR or \$address_object_name
destination	String	Destination CIDR or \$address_object_name
service	String	\$service_object_name or bare identifier
protocol	String	IP protocol (e.g. tcp, udp)

11.18 build_qos_policy

Attaches Quality of Service policies to matched nodes. Each policy defines a set of traffic classes (classifier, marker, scheduler, policer, shaper) and a set of interface bindings that apply policies in the ingress or egress direction.

Table 27. build_qos_policy fields.

Field	Type	Required	Description
selector	String	Yes	NETCFG query for target nodes
policies	map (name → QoSPolicy)	No	Named QoS policies
interfaces	map (name → binding)	No	Per-interface policy bindings

A QoSPolicy contains a map of named TrafficClass entries. Each traffic class may specify a classifier (matching on DSCP, CoS, protocol, addresses, or ports), a marker (rewriting DSCP/CoS), a scheduler (priority queuing, weighted fair queuing, bandwidth guarantees), a policer (CIR/burst enforcement), and a shaper (output rate limiting). An InterfaceBinding specifies an input_policy and/or output_policy name.

Example: data-centre QoS policy

```
- type: build_qos_policy
  selector: "nodes[role=='leaf']"
  policies:
    DC-QOS:
      classes:
        voice:
          classifier: { dscp: "ef" }
          scheduler: { priority: true, bandwidth_percent: 10 }
          policer: { cir: 1000000, exceed_action: drop }
        bulk:
          classifier: { dscp: "af11" }
          scheduler: { weight: 20 }
          shaper: { shape_rate: 500000000 }
  interfaces:
    Ethernet1:
      input_policy: DC-QOS
      output_policy: DC-QOS
```

Note

The device_os annotation determines platform-specific QoS semantics (fully supported across Cisco NX-OS, Cisco IOS-XR, Arista EOS, Juniper JunOS, Nokia SR OS, VyOS, and general Cisco IOS). For example, NX-OS devices enforce a maximum of 8 traffic classes per policy and require explicit system-qos activation.

11.19 build_vxlan

Assigns VXLAN tunnel bindings to matched nodes. VNIs are allocated sequentially from vni_base.

Table 28. build_vxlan fields.

Field	Type	Required	Description
selector	String	Yes	NETCFG query for VTEP nodes
vni_base	u32	Yes	Starting VNI (e.g. 10000)
mcast_group_base	String	No	Multicast group for BUM replication

11.20 build_evpn

Assigns EVPN route-distinguisher (RD) and route-target (RT) values to matched nodes.

Table 29. build_evpn fields.

Field	Type	Required	Description
selector	String	Yes	NETCFG query for EVPN nodes
route_distinguisher_base	String	Yes	RD base (e.g. auto)
route_target_base	String	Yes	RT base (e.g. 65001:10000)

11.21 tag_nodes

Stamps flat key-value metadata onto matched nodes. Tags are merged into each node's data_json and are available to subsequent selectors and assertions.

Table 30. tag_nodes fields.

Field	Type	Required	Description
selector	String	Yes	NETCFG query for target nodes
tags	map (string → value)	Yes	Key-value pairs to stamp

11.22 map_hardware_inventory

Assigns chassis model and line-card slot mappings to matched nodes. Used by the TSDM layer (§17.4) via NQL extension functions (e.g., hw.port_name(node.chassis, slot)) to resolve abstract interface names into physical slot/port identifiers.

Table 31. map_hardware_inventory fields.

Field	Type	Required	Description
selector	String	Yes	NETCFG query for target nodes
chassis_model	String	Yes	Chassis identifier (e.g. dcs-7508)
slots	map (integer → string)	Yes	Slot → line-card model mapping

11.23 assert_state

A mid-pipeline assertion checkpoint. Syntactically identical to a top-level AssertionSpec but placed inline within a layer's primitive list. This allows the operator to validate intermediate state between primitives rather than waiting until the post-execution assertion pass.

Table 32. assert_state fields.

Field	Type	Required	Description
name	String	Yes	Assertion name (for diagnostics)
severity	String	No	error (default) or warning
select	String	Yes	NETCFG query for items to check
check	AssertionCheck	Yes	Predicate (same types as top-level assertions)
help	String	No	Remediation hint shown on failure

12 Design-rule assertions

Design plans may declare assertions: post-execution invariants checked after all primitives complete but before commit. These assertions complement NTE's structural invariant checks (NTE-TR-001 §Policy Engine), which enforce correctness-by-construction at the graph layer. Each assertion has a severity: error (pipeline failure) or warning (diagnostic only).

Listing 7. Design-rule assertions in a design plan.

```

assertions:
- name: "all routers have hostname"
  severity: error
  select: all()
  check: { type: field_exists, field: hostname }
- name: "IPs within allocation"
  severity: error
  select: "nodes[src_ip != null]"
  check:
    field_in_cidr:
      field: src_ip
      cidr: "10.0.0.0/8"
- name: "unique hostnames per site"
  severity: warning
  select: all()
  check:
    unique_per_group:
      field: hostname
      group_by: site
- name: "private ASN range"
  severity: warning
  select: "nodes[asn != null]"
  check:
    custom_query:
      expression: "asn >= 64512 && asn <= 65534"

```

Nineteen check types are available:

Table 33. Assertion check types.

Check	Semantics
field_exists	Every matched node has the named field in its data_json
min_count	The matched set contains at least N items
max_count	The matched set contains at most N items
min_edges	Every matched node has at least N outgoing edges
unique_per_group	Within groups defined by group_by, the field value is unique
field_in_cidr	Every matched node's IP field is contained within the specified CIDR
custom_query	Arbitrary NETCFG query expression evaluated per matched entity; fails if false
use_rule	Reference a named NQL macro from the rules: section
is_connected	The matched subgraph (nodes + interconnecting edges) is connected
is_acyclic	The matched subgraph contains no cycles
is_bipartite	The matched subgraph is bipartite (two-colourable)
reachability	Every matched node can reach at least one node in a target selector
match_schema	Each matched node's data_json validates against a JSON Schema
zone_membership	Every matched node belongs to one of the declared security zones
zone_policy_exists	A zone policy exists for the specified from/to zone pair
no_shadowed_rules	No rule in a zone policy is fully shadowed by a preceding rule
zone_pairs_covered	All declared zone pairs have corresponding zone policies
zone_policy_permits	A specific traffic 5-tuple is permitted by the zone policy
zone_policy_denies	A specific traffic 5-tuple is denied by the zone policy

This enables “policy as code” directly in the design plan. Assertions are evaluated cross-layer (no layer filter), so a single assertion can validate properties across both physical and protocol layers.

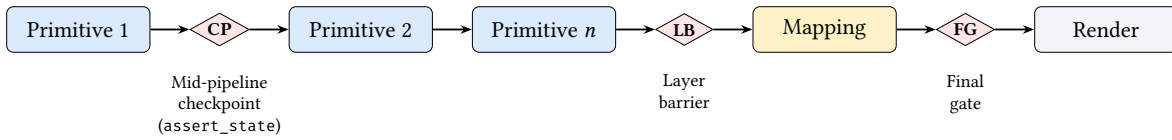


Figure 14. Assertion firing points. **CP**: mid-pipeline checkpoint (inline `assert_state`); **LB**: layer barrier (runs after a layer’s primitives complete); **FG**: final gate (runs before rendering). All three evaluate the same assertion types but fire at different pipeline positions for earlier feedback.

13 Named rules (NQL macros)

The `rules` section defines named NQL macros that can be referenced by assertions via the `use_rule` check type. This avoids duplicating complex NETCFG query expressions across multiple assertions.

Listing 8. Named rules and their use in assertions.

```

rules:
  tenant-assigned: "has(node.tenant) && node.tenant != ''"
  vrf-configured: "has(node.vrf_name) && node.vrf_name != ''"
  leaf-has-vlan: "has(node.vlan_id) && node.vlan_id > 0 && node.vlan_id < 4095"

```

Rules are simple string-valued NETCFG query expressions stored as a name-to-expression map. At evaluation time, a `use_rule` check looks up the named rule and evaluates it as though it were an inline `custom_query` expression. Rules from imported files are merged into the importing design plan’s rule namespace.

14 Blast radius policies

The `blast_radius` section declares change-safety policies that constrain how much the topology may change in a single commit. Each policy specifies a check against the computed `MutationPlan`. This calculation goes beyond static property matching by performing an automated blast-radius impact analysis. It actively traces topological dependencies via the underlying graph engine, identifying adjacent nodes across logical overlays (like BGP and OSPF) to quantify both direct and indirect operational risks.

Table 34. Blast radius check types.

Check	Semantics
<code>max_deletions</code>	At most N nodes or edges may be deleted
<code>max_modifications</code>	At most N nodes or edges may be modified
<code>max_percentage_changed</code>	At most $P\%$ of the topology may be altered (0.0–100.0)

Table 35. BlastRadiusSpec fields.

Field	Type	Required	Description
<code>name</code>	String	Yes	Policy name (for reporting)
<code>select</code>	String	No	NETCFG query to scope the policy; omit for global
<code>check</code>	object	Yes	One of the check types above, with a <code>value</code> field

When `select` is omitted, the policy applies to the entire mutation plan.

Listing 9. Blast radius policies.

```

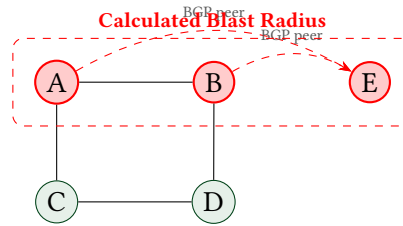
blast_radius:
  - name: tenant-vrf-protection
    check:
      type: max_deletions

```

```

count: 3
- name: fabric-change-cap
check:
  type: max_percentage_changed
  value: 25.0

```



Automated analysis traces both physical and logical dependencies (e.g., BGP overlays) to quantify the total operational risk of a node change.

Figure 15. Automated Blast-Radius impact analysis. The engine identifies that changing nodes A or B directly impacts logical peer E, expanding the risk profile beyond the physical link boundaries.

15 Target-specific transforms

The `transforms:` section declares AST-to-AST rewrite rules that adapt vendor-neutral stanzas into platform-specific syntax (§17.4 details the execution engine). Each `TransformationSpec` has a name, an optional when predicate (`NETCFG` query expression evaluated per device), and a sequence of `RewriteRule` entries:

Table 36. Transform rewrite rule fields.

Field	Type	Required	Description
name	String	Yes	Transform name
when	String	No	NQL predicate (e.g. <code>device_os == 'nxos'</code>)
rules	Vec<RewriteRule>	Yes	Sequence of match/apply rewrites

Each `RewriteRule` has a `match_expr` (`NETCFG` query expression selecting stanzas) and an `apply` map (field name → NQL expression computing the new value):

Listing 10. Vendor-specific transform.

```

transforms:
- name: nxos-interface-rewrite
  when: "device_os == 'nxos'"
  rules:
  - match_expr: "stanza.kind == 'interface'"
    apply:
      name: "'Ethernet1/' + string(stanza.fields.abstract_index + 1)"

```

ENGINE IMPLEMENTATION (DEEP DIVE)

16 Pipeline Internals

This section details the mechanics of each pipeline layer introduced in §5; Figure 11 shows the overall data flow.

16.1 Layer 1: Intent Transformation

The design plan is parsed into an AST and its primitives are applied to a transactional *shadow* copy of the annotated topology (Figure 16). NTE-TR-001 calls this “Shadow Topology Isolation”: a copy-on-write clone created via serialisation round-trip. If any primitive or assertion fails, the shadow

is discarded and the production graph remains untouched. Each primitive's selectors reference annotations already present on the topology nodes (e.g. role, site, device_os); the primitive then derives new state from those annotations and writes it back to the shadow.

Composable primitives execute in declaration order within each layer. Topology primitives (mesh_nodes, build_protocol_layer) create graph structure; allocation primitives (provision_ips, allocate_resources) assign derived identifiers; policy primitives (build_zone_policy, build_routing_policy) compile intent into per-device rule objects. Each primitive follows the *select* → *enrich* → *validate* pattern (§2.7), and each sees the enriched model left by its predecessors.

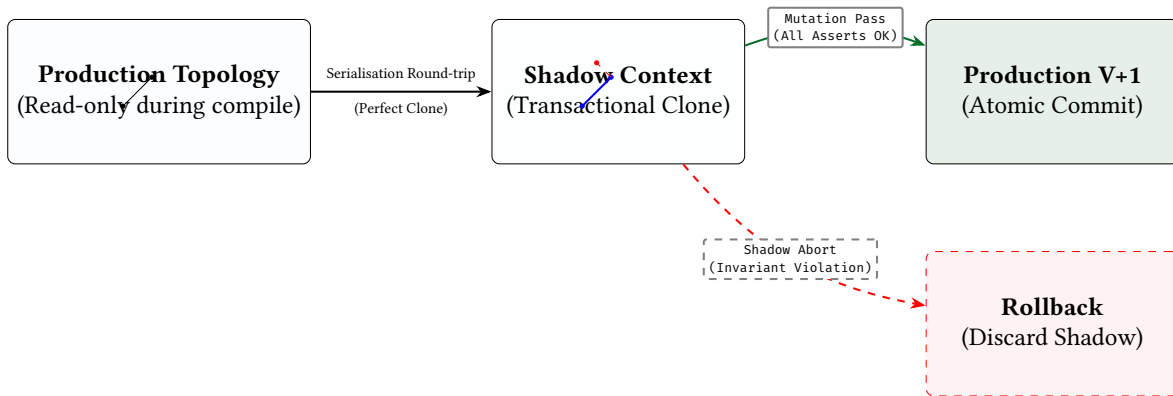


Figure 16. Transactional Safety via the Shadow Topology. Mutations occur on an isolated clone. If any primitive or assertion fails, the shadow is discarded without side-effects on the production graph.

16.2 Layer 2: Topological Assertions

After all primitives complete, the DSL-driven assertion engine (§12) evaluates every assertion declared in the design plan against the enriched graph. Assertions are typed checks—field existence, count bounds, CIDR containment, graph connectivity, zone-pair coverage, and arbitrary NQL predicates—evaluated cross-layer so that a single assertion can span physical and protocol layers simultaneously.

Assertions with severity error abort the pipeline; those with severity warning emit diagnostics but allow compilation to continue. This layer acts as the quality gate between intent transformation and lowering: only a fully validated model proceeds to DeviceIR generation.

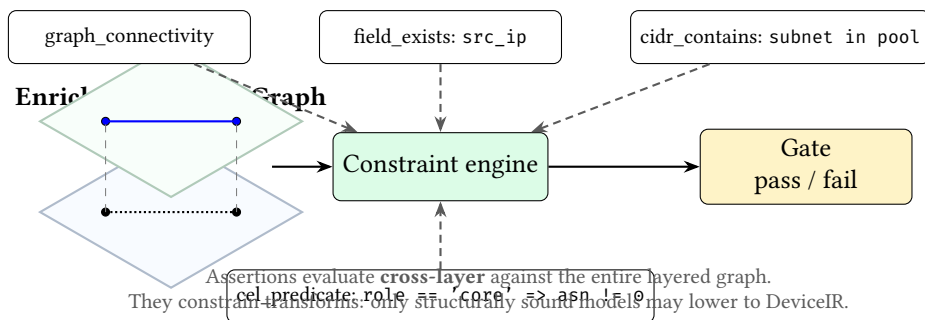


Figure 17. Assertions as a semantic gate. Rather than “tests after generation”, assertions are part of the compilation pipeline: they use the NETCFG query language to evaluate cross-layer and constrain which enriched graphs are allowed to lower into DeviceIR.

16.3 Layer 3: DeviceIR and the Mapping DSL

Between stage 2 (Assertions) and stage 4 (TSDM) sits the Mapping Evaluation engine. A Mapping DSL lowers the rich topology graph into a vendor-neutral **Abstract Syntax Tree (DeviceIR)**. This representation captures logical intent (e.g. “Neighbor X is remote-as Y”) without hardware context.

A mapping rule consists of a selector and a stanza template. When evaluated against the graph, it emits discrete configuration units (e.g., interfaces, BGP neighbours) that are attached to each node's `DeviceContext.stanzas` list (Figure 18).

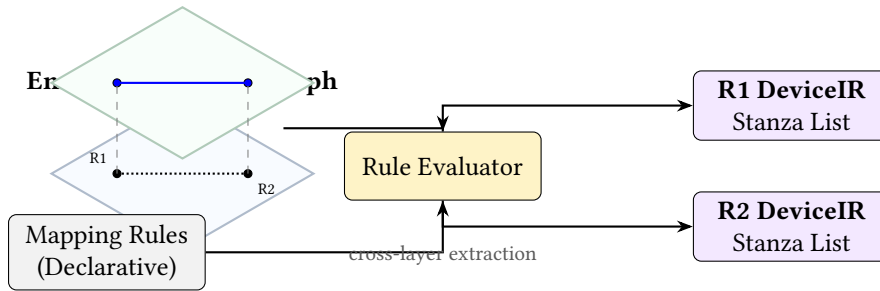


Figure 18. Mapping evaluation. The rule evaluator uses the `NETCFG` query language to query the rich, multi-layer graph stack and extracts flat, device-centric DeviceIR structures for each node (R1, R2), collapsing the topology into discrete configuration units.

Listing 11. Mapping DSL extracting an interface stanza.

```
rules:
- selector: "edges[layer=='physical']"
  stanza:
    kind: "interface"
    fields:
      name: "{{ node.peer_interfaces[peer.hostname] }}"
      description: "Link to {{ peer.hostname }}"
      mtu: 9000
```

16.4 Layer 4: Target-Specific Transform (TSDM)

The **TSDM Layer** performs AST-to-AST transformations (Figure 19). It lowers the Neutral AST into a platform-specific AST, remapping logical interfaces to physical ports (e.g. `Eth1` → `Eth1/1/1`) based on the target's chassis model and hardware layout.

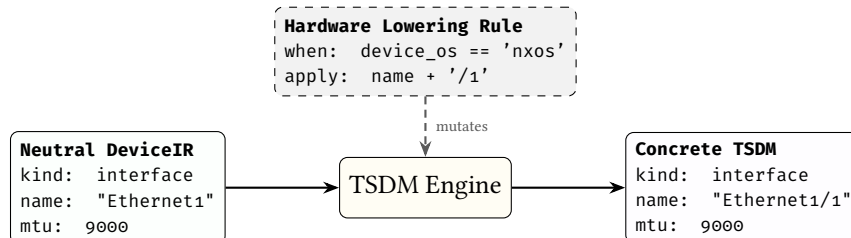


Figure 19. AST-to-AST lowering via TSDM. Vendor-neutral DeviceIR is transformed into a hardware-specific data model by the TSDM engine, which executes declarative mutation rules against the AST.

16.5 Layer 5: Syntax Serialisation

MiniJinja templates act as a *dumb emission layer*, serialising the final Target-Specific AST into vendor-specific CLI syntax. Templates contain no conditional logic beyond iteration over stanza lists and field interpolation—all architectural decisions have already been resolved by the preceding layers. This keeps templates thin and auditable: a template bug can only affect formatting, never topology or policy semantics.

The `TransactionalOutput` writer collects all files in temporary locations (using `NamedTempFile` in the same directory for same-filesystem guarantee), then atomically renames all via `POSIX rename(2)` on commit. If any stage fails, all temp files are dropped without committing—no partial output directory is ever visible.

Three artifacts are emitted per node:

- `{hostname}.conf`: the rendered device configuration.

- `{hostname}.deviceir.json`: the raw DeviceIR stanzas (audit).
- `{hostname}.lineage.json`: per-line mapping from config lines to the `rule_id` that produced them (provenance tracing).

16.5.1 Determinism guarantee

Given a fixed annotated topology G , design plan B , and mapping M , the output configuration set $C = \text{compile}(G, B, M)$ is deterministic. Serialisation round-trip produces a bit-identical shadow. Primitives execute in declaration order. Selectors evaluate deterministically. Node ID allocation is monotonic. IP allocation iterates edges sorted by (src, dst) . DeviceIR stanzas use BTreeMap and are post-sorted. Template rendering is pure.

17 Core Mechanisms

17.1 Declarative IPAM and Resource Allocation

The `NETCFG` IP allocator automates the derivation of edge and node addresses from logical CIDR pools. Unlike traditional IPAM where addresses are manually assigned, `NETCFG` uses *sticky functional allocation*: the IP is a derived fact of the topology’s cable relationships.

The allocator implements a five-pass engine to ensure determinism and idempotency:

1. **State discovery**: Scan all nodes for existing IP assignments (`src_ip`, `dst_ip`, `subnet` fields).
2. **Edge selection**: Evaluate the `NETCFG` query; sort matched edges by (src, dst) for determinism.
3. **Pre-reservation**: Insert existing allocations into the allocator, preventing re-allocation and enforcing stickiness.
4. **Fulfilment**: For each edge, allocate a subnet using the configured strategy (dense, sparse, or hash). Extract host addresses from the subnet.
5. **Write-back**: Write `src_ip`, `dst_ip`, `subnet` (and IPv6 equivalents for dual-stack) into each endpoint’s `DeviceContext`.

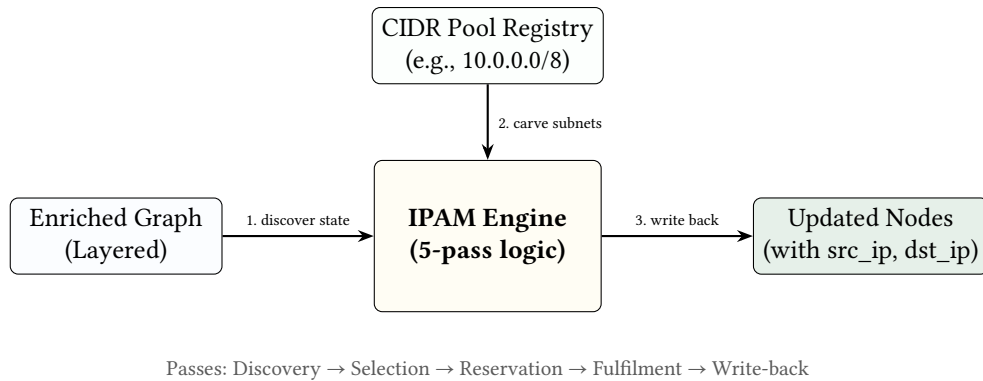


Figure 20. Declarative IPAM workflow. The engine queries the topology graph to discover existing state, carves subnets from the central registry according to design rules, and mutates the layered graph with deterministic host addresses.

Identity-based stickiness. The allocator uses a canonical identity —`sorted(hostname_a, hostname_b)`—as the primary key. If a prior allocation exists for the same identity, the same prefix is returned without pool search. This anchors allocations to stable semantic identity, not integer IDs. If topology IDs change (e.g., after a re-import) but hostnames remain the same, allocations are preserved.

VRF isolation. The allocator maintains per-VRF state. Two primitives using the same CIDR pool in different VRFs can allocate overlapping subnets without conflict. Pool overlap *within* the same VRF is detected and rejected.

Allocation strategies:

- **dense**: Sequential allocation from the pool start. First edge gets the first available subnet.

- **sparse:** Skips every other subnet, leaving growth headroom between allocations.
- **hash:** Subnet position is derived from a hash of the edge identity, producing topology-independent allocations that do not depend on evaluation order.

Hierarchical pools. A `global_pool` primitive registers a large CIDR (e.g., `10.0.0.0/8`). Subsequent `provision_ips` primitives reference the pool by name and specify a `pool_size` (e.g., 24), carving /24 blocks from the parent pool on demand.

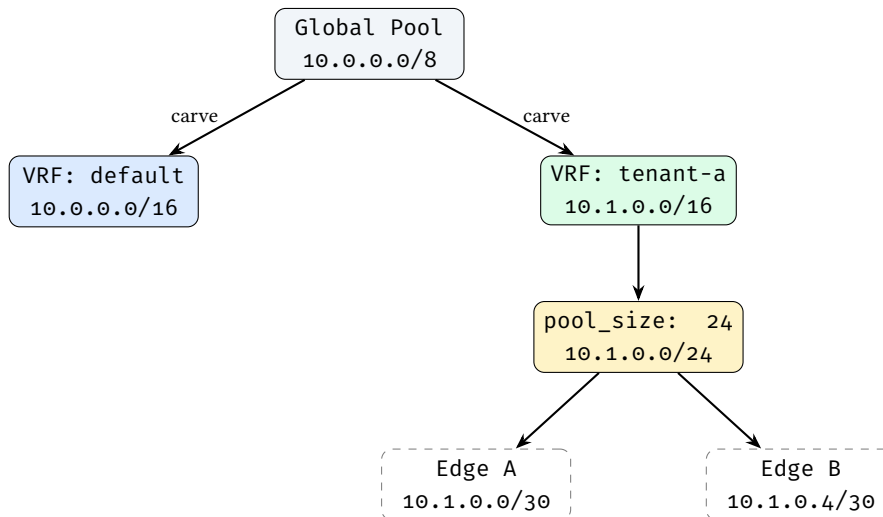


Figure 21. Hierarchical IP pool carving with VRF isolation.

Dual-stack. When `ipv6_pool` is specified, both IPv4 and IPv6 addresses are allocated in a single primitive invocation. IPv6 fields (`src_ipv6`, `dst_ipv6`, `subnet_v6`) are written alongside IPv4 fields.

17.2 Protocol overlay construction (`build_protocol_layer`)

`build_protocol_layer` clones physical nodes into a named protocol layer. For each selected source-layer node:

1. Allocate a new monotonic node ID.
2. Add the node to the target layer.
3. Deep-merge the config overlay into the cloned node's metadata.
4. Record `_source_id` as a parent mapping. For MLAG/vPC setups, `_source_id` can be an array mapping one logical protocol node to multiple physical host switches.
5. Optionally (`clone_underlying: true`) clone edges between source-layer nodes into the overlay.

Cross-layer traceability: the `_source_id` field links each protocol node to its physical origin. Since `provision_ips` already enriched the physical node's DeviceContext with IP addresses, the cloned protocol node inherits them via deep merge—this is how IP addresses flow from IPAM allocation to BGP configuration without explicit wiring.

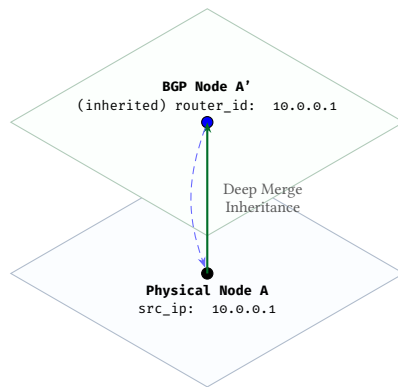


Figure 22. Cross-layer state inheritance. Because the protocol node stores a reference to its physical parent, state allocated in Layer 1 (IPAM) automatically flows into Layer 2 (BGP) without the operator needing to manually pass variables between primitives.

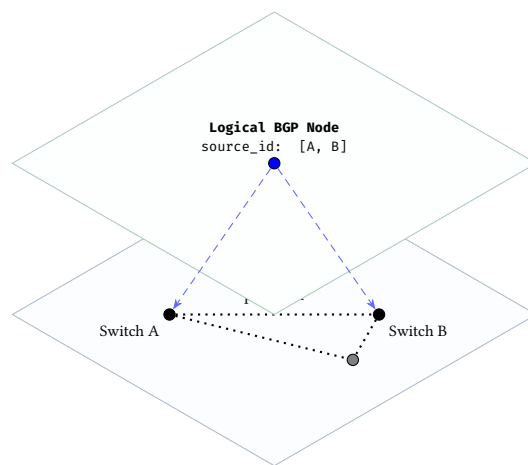


Figure 23. MLAG/vPC topology mapping. Two physical nodes collapse into a single logical protocol node via array `_source_id`, representing a multi-chassis cluster as a single routing entity.

Deep merge semantics: When both the base node and the config overlay contain nested JSON objects under the same key, the merge recurses key-by-key (overlay wins on conflict). Arrays are replaced entirely rather than concatenated, making it idempotent. This preserves sibling keys that the overlay does not mention. For example, a base node with `{bgp: {as: 65000, holdtime: 180}}` and an overlay with `{bgp: {timers: {keepalive: 60}}}` produces `{bgp: {as: 65000, holdtime: 180, timers: {keepalive: 60}}}`.

Idempotency: The primitive scans the target layer for existing nodes with `_source_id` matching source nodes. Already-cloned nodes are skipped, so re-running the primitive after a partial failure produces the correct result.

17.3 Configuration diffing (DiffEngine)

The diff engine computes a *mutation plan* between two topologies by exporting each layer to JSON and comparing:

1. **Node diff:** Per layer, compare node sets by a composite semantic key (`layer`, `hostname`, `protocol_type`). Diffing by integer ID is avoided, as node insertions would shift downstream IDs and create massive false diffs. Detect additions, deletions, and modifications (field-by-field property comparison via `extract_properties`, which unpacks the `data_json` blob).
2. **Edge diff:** Compare by a composite semantic key derived from the resolved (`src`, `dst`) node semantic keys, allowing structural property comparison without relying on volatile internal edge IDs.

The `MutationPlan` contains six change types: `AddNode`, `RemoveNode`, `UpdateNode`, `AddEdge`, `RemoveEdge`, `UpdateEdge`. Changes are sorted deterministically by `(change_type, layer, id/src/dst)` for reproducible output.

The `plan` command displays the plan as a human-readable diff; the `ci-run` command uses it to compute per-device configuration diffs.

17.4 AST Transformations (TSDM Layer)

The **Target-Specific Device Model (TSDM)** layer implements the compiler's lowering phase via DSL-driven **AST-to-AST transformations**, whose design plan syntax is described in §15. This section details the execution engine that bridges logical design and physical hardware constraints.

17.4.1 The Transformation DSL

Lowering rules are defined in the design plan using an expressive rewrite-rule grammar. Each rule consists of a `match_expr` (NQL predicate) and an `apply` map (field mutations).

Example: NX-OS interface lowering

```
transforms:
- name: "nxos_lowering"
  when: "device_os == 'nxos'"
  rules:
  - match_expr: "kind == 'interface' && name.startsWith('Ethernet')"
    apply:
      name: "name + '/1'" # Ethernet1 -> Ethernet1/1
      mtu: "9216"         # Force jumbo frames
```

17.4.2 Execution Logic

The transformation engine performs a deep-walk of the `DeviceIR` stanzas. For each stanza, it:

1. Binds the stanza's fields as NQL variables.
2. Evaluates the `match_expr` predicate.
3. If matched, executes the `apply` expressions to mutate the stanza's state.

This structured approach allows complex hardware logic—such as mapping logical ports to specific line-card slots in a modular chassis—to be expressed as programmable transformations rather than hardcoded logic or brittle templates.

The key architectural insight is that `DeviceContext` accumulates state across the entire primitive pipeline:

1. `mesh_nodes`: writes `mesh_type`, `peer_count`, and optionally `peer_interfaces` into metadata.
2. `provision_ips`: writes `src_ip`, `dst_ip`, `subnet`, `vrf`.
3. `build_protocol_layer`: deep-merges all of the above into the cloned protocol node, plus the config overlay.
4. `allocate_resources`: writes the allocated resource value (e.g., `asn`) into metadata.

By the time Stage 4 (rendering) occurs, each node's `DeviceContext` carries hostname, IPs, protocol metadata, interface assignments, and any custom fields—the complete information needed to produce a device configuration file.

18 CLI Reference

`NETCFG` provides twelve CLI commands spanning the development-to-deployment lifecycle. All commands use `clap` for argument parsing. Table 37 shows which pipeline layers each command invokes.

Table 37. CLI commands and the pipeline layers they invoke. ✓ = executes; ◦ = optional (via flag).

Command	<i>L1 Intent</i>	<i>L2 Assert</i>	<i>L3 Mapping</i>	<i>L4 TSDM</i>	<i>L5 Render</i>	Primary output
generate	✓	✓	✓	✓	✓	Device config files
validate	✓	✓			◦	Pass/fail exit code
plan	✓	✓			◦	Mutation diff
inspect	◦					AST / DOT / trace
ci-run	✓	✓	✓	✓	✓	JSON report
lint	✓	✓				Assertion results
impact	✓	✓				Risk assessment
report	✓	✓				Markdown report
fmt						Formatted YAML
import						.nte archive
export-clab	✓	✓				clab.yaml
sync						.nte archive / stdout

18.1 netcfg generate

Runs the full five-layer compilation pipeline (§5), writing artefacts to disk.

```
netcfg generate <topology> <design plan> \
  --templates-dir <DIR> \
  [--output-dir <DIR>] [--emit-ir] [--verbose]
```

Argument	Default	Description
topology	—	Path to annotated topology archive (.nte) or NSoT URI
design plan	—	Path to design plan YAML
—templates-dir	—	Directory containing .j2 templates
—output-dir	./output	Output directory for generated files
—emit-ir	false	Also write {hostname}.ir.json
—verbose	false	Print per-node rendering progress

Output: one .cfg file per device (plus .ir.json if —emit-ir). Absolute paths are printed to stdout, one per line, suitable for piping.

18.2 netcfg validate

Executes the pipeline in dry-run mode: all primitives run with `dry_run=true`, skipping DataFrame writes but verifying what *would* change. Optionally pre-flights templates by rendering to memory.

```
netcfg validate <topology> <design plan> \
  [--templates-dir <DIR>] [--verbose]
```

Exit codes: 0 on success, non-zero on validation failure (parse errors, assertion failures, template syntax errors, IP pool conflicts).

18.3 netcfg plan

Computes the topology MutationPlan (current vs. desired) and displays additions, removals, and property changes. With `-diff`, it renders before/after configs and shows a unified text diff per device using the similar crate.

```
netcfg plan <topology> <design plan> \  
  [--diff] [--templates-dir <DIR>] [--verbose]
```

Output without `-diff`: colourised plan (green for additions, red for removals, yellow for modifications). Summary line: `+N additions, -N removals across M layers`.

Output with `-diff`: unified diff per device, preceded by `diff -cfg {hostname}`.

18.4 netcfg inspect

Provides three views into intermediate state:

```
netcfg inspect <path> --ast           # Print design plan AST  
netcfg inspect <path> --topology      # Export as Graphviz DOT  
  [--design plan <FILE>]  
netcfg inspect <path> --trace <NODE_ID> # Trace node state changes  
  --design plan <FILE>
```

- `-ast`: Parses the file as a design plan and prints the AST tree.
- `-topology`: Exports the graph as Graphviz DOT. With `-design plan`, applies the design plan first. Output includes per-layer subgraph clustering, hostname labels, and subnet annotations.
- `-trace <node_id>`: Replays primitive execution and prints before/after DeviceContext snapshots for the specified node at each primitive step. When design plans fail with selector or assertion errors, finding the root cause in a 1,000-node graph is challenging; `-trace` is the primary debugging workflow for design plan authors.

18.5 netcfg ci-run

The CI/CD integration point. Runs validate, plan, and config diff in a single invocation.

```
netcfg ci-run <topology> <design plan> \  
  --templates-dir <DIR> [--json]
```

With `-json`, emits a structured payload:

```
{  
  "status": "success",  
  "topology_additions": 42,  
  "topology_removals": 0,  
  "configuration_diffs": {  
    "core-0": "- ip address 10.0.0.4/30\n+ ip address 10.0.0.8/30"  
  },  
  "warnings": []  
}
```

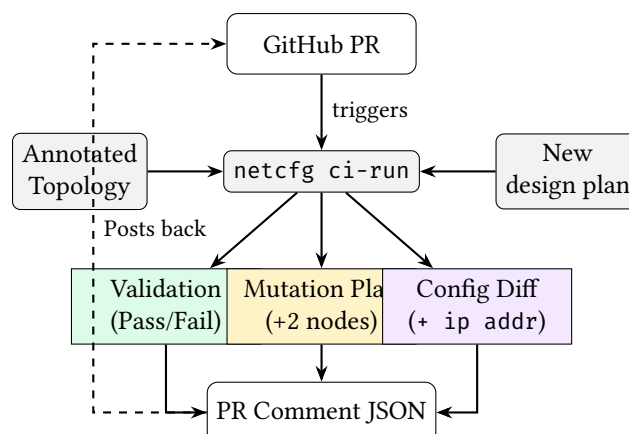


Figure 24. The NetDevOps CI/CD pipeline enabled by `ci-run`.

This integrates with GitHub Actions, GitLab CI, or any CI system that can parse JSON.

18.6 netcfg import

Imports a YAML topology definition into an NTE archive.

```
netcfg import <yaml_path> <output.ntc>
```

The importer reads a topogen-format YAML file (defining nodes with types, layers, and properties) and serialises it to an .ntc archive that other commands consume.

18.7 netcfg lint

Runs design-rule assertions without the full validation pipeline. Applies the design plan in dry-run mode, then evaluates assertions and reports results.

```
netcfg lint <topology> <design plan> [--verbose]
```

Returns assertion results with pass/fail status, severity, and diagnostic messages. Exit code 1 only when error-severity assertions fail; warnings are reported but do not cause failure.

18.8 netcfg fmt

Standardises design plan YAML formatting. Parses the design plan, re-serialises it in canonical form, and compares byte-for-byte. Idempotent.

```
netcfg fmt <design plan> [--check] [--verbose]
```

With `--check`, exits with code 1 if formatting changes are needed (useful as a pre-commit hook or CI gate). Without `--check`, overwrites the file in place.

18.9 netcfg impact

Calculates the operational blast radius of a design plan change against a base topology. The topology argument accepts either a local .ntc archive path or an NSoT URI (e.g. `netbox://host?site=ldn1`); see `netcfg sync` (§18.12) for URI syntax.

```
netcfg impact <topology> <design plan> [--detail] [--verbose]
```

The `--detail` flag expands the low-risk section to list individual nodes (by default only a count is shown).

For each mutation in the computed plan, assigns a risk level:

Risk	Colour	Trigger
High	Red	Node removal; critical property change (asn, router_id, src_ip, subnet); physical link add/remove
Medium	Yellow	Node addition; logical link add/remove
Low	Green	Metadata-only update

Critical properties are: `asn`, `router_id`, `src_ip`, `dst_ip`, `subnet`, `loopback`, `mgmt_ip`. Output includes per-node risk attribution with reasons.

18.10 netcfg report

Generates an automated Markdown architecture report from a compiled topology.

```
netcfg report <topology> <design plan> -o <output.md> [--verbose]
```

The report contains three sections:

1. **Node inventory:** hostname, type, OS, ASN, loopback, management IP.
2. **Physical links & IP addressing:** source/destination hosts, interfaces, IPs, subnets.
3. **Topology visualisation:** A `netviz-json` code block with a custom JSON schema (`NetVizNode`, `NetVizEdge`, `NetVizTopology`) suitable for rendering by external visualisation tools.

18.11 netcfg export-clab

Generates a Containerlab `clab.yaml` simulation file from a compiled topology, enabling rapid lab deployment for testing (e.g. the three-site mesh from §19). The topology argument accepts a local `.nte` archive or an NSoT URI (§18.12).

```
netcfg export-clab <topology> <design plan> \
-o <clab.yaml> [--configs-dir <DIR>] [--verbose]
```

Heuristically maps `device_os` to Containerlab container kinds: `cisco_iosxr` → `vr-xrv9k`, `nokia_srl` → `srl`, `arista_eos` → `ceos`, `default` → `linux`. When `--configs-dir` is provided, bind-mounts generated configuration files into OS-specific remote paths (e.g. `/etc/opt/srlinux/config.json` for SRL, `/mnt/flash/startup-config` for cEOS).

18.12 netcfg sync

Synchronises topology data with external Network Source of Truth (NSoT) systems. Two subcommands are available:

```
netcfg sync pull <uri> [-o <output.nte>] [--token <TOKEN>] [--insecure]
netcfg sync push <uri> <source> [--token <TOKEN>]
```

`sync pull` fetches a live topology from an NSoT and converts it into an NTE topology archive. The URI scheme identifies the provider: `netbox://host[:port]?site=X®ion=Y` targets a Net-Box [6] instance via its GraphQL endpoint; query parameters are passed as filters. Authentication is resolved in order: the `--token` flag, then the `NETBOX_TOKEN` environment variable. When `-o` is supplied the topology is persisted to disk; otherwise a summary (node and edge counts) is printed. The `--insecure` flag disables TLS certificate verification for self-signed development instances.

`sync push` is a planned stub that will write intent back to the NSoT (see §22).

Note: NSoT URI in other commands

The topology argument of `impact` (§18.9) and `export-clab` (§18.11) also accepts a `netbox://` URI. Internally these commands dispatch through the same `load_topology_source()` helper, so a live NSoT topology can be used without a local archive.

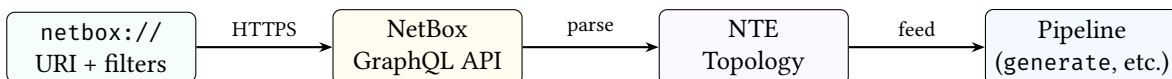


Figure 25. NSoT synchronisation data flow.

19 End-to-End Worked Example

We trace the `three-site-mesh.yaml` design plan through every pipeline stage. The six stages below correspond to the five pipeline layers in §5: Stages 1–3 are subsumed by Layer 1 (Intent Transformation), Stage 4 maps to Layer 3 (DeviceIR), Stage 5 to Layer 5 (Serialisation), and Stage 6 is the atomic write that concludes Layer 5.

The annotated topology has 12 routers across 3 sites (4 per site: `site-a-r1` through `site-c-r4`), each annotated with `role`, `site`, and `device_os` properties that the design plan’s selectors reference.

19.1 Input: design plan

Listing 12. `three-site-mesh.yaml` (abridged).

```
version: 1
layers:
  # Stage 1: Full mesh within each site
  - name: input
    primitives:
      - type: mesh_nodes
        selector: "nodes[site='a']"
        mesh_type: full
```

```

- type: mesh_nodes
  selector: "nodes[site='b']"
  mesh_type: full
- type: mesh_nodes
  selector: "nodes[site='c']"
  mesh_type: full

# Stage 2: IP allocation per site
- name: input
  primitives:
    - type: provision_ips
      selector: "edges[src>=0]"
      pool: "10.1.0.0/24"
      subnet_size: 30
      strategy: dense

# Stage 3: BGP overlay
- name: input
  primitives:
    - type: build_protocol_layer
      selector: "nodes[true]"
      layer: bgp
      config:
        protocol_type: bgp
        asn_base: "65000"

```

19.2 Stage 1: Parse and validate

The parser validates the YAML, confirms version: 1, and compiles each selector string (nodes[site='a'], edges[src>=0], etc.) to a query program. The normaliser converts site='a' to site=="a".

19.3 Stage 2: Shadow topology

The executor clones the annotated topology via a serialisation round-trip, creating an isolated shadow copy. All subsequent mutations operate on the shadow; the source topology is never modified.

19.4 Stage 3: Primitive execution

mesh_nodes (three invocations): Creates $3 \times \binom{4}{2} = 18$ edges (6 per site). The selector nodes[site='a'] matches the 4 nodes with {site: "a"} in their data_json. Each node gets mesh_type: "full" and peer_count: 3 written to its metadata.

provision_ips: The selector edges[src>=0] matches all 18 edges. Edges are sorted by (src, dst). The five-pass algorithm allocates /30 subnets from 10.1.0.0/24:

- Edge (1,2): subnet 10.1.0.0/30, IPs 10.1.0.1 and 10.1.0.2
- Edge (1,3): subnet 10.1.0.4/30, IPs 10.1.0.5 and 10.1.0.6
- ...continuing through all 18 edges.

After this stage, node site-a-r1 has: src_ip: "10.1.0.1", subnet: "10.1.0.0/30", vrf: "default".

build_protocol_layer: Clones all 12 physical nodes into the bgp layer (new IDs 13–24). Each clone receives:

- protocol_type: "bgp" and asn_base: "65000" (from the config overlay)
- _source_id: 1 (parent mapping)
- All properties from the physical node, including the IPs written by provision_ips (via deep merge)

19.5 Stage 4: Mapping evaluation

The companion mapping (three-site-mesh-mapping.yaml) extracts per-device stanzas from the enriched graph:

```

rules:
- selector: "edges[layer=='physical']"

```

```

stanza:
  kind: "interface"
  fields:
    name: "{{ node.peer_interfaces[peer.hostname] }}"
    description: "Link to {{ peer.hostname }}"
    ip_address: "{{ node.src_ip }}"
    mtu: 9000
- selector: "nodes[layer=='bgp']"
stanza:
  kind: "bgp"
  fields:
    local_asn: "{{ node.asn }}"
    router_id: "{{ node.loopback }}"
    neighbors: "{{ node.bgp_peers }}"

```

The evaluator walks each rule’s selector, matches the relevant nodes/edges, and emits one stanza per match. Each stanza carries provenance: `{rule_id: "rule_0"}`, linking rendered output back to its source rule.

19.6 Stage 5: Render

Stanzas are grouped by hostname. The Jinja2 template iterates over each node’s stanza list and emits vendor-specific CLI syntax. For `site-a-r1`, the rendered output includes:

```

! site-a-r1.conf (abridged)
interface Ethernet0
  description Link to site-a-r2
  ip address 10.1.0.1/30
  mtu 9000
!
interface Ethernet1
  description Link to site-a-r3
  ip address 10.1.0.5/30
  mtu 9000
!
router bgp 65001
  router-id 10.255.0.1
  neighbor 10.1.0.2 remote-as 65001
  neighbor 10.1.0.6 remote-as 65001

```

Each of the 12 routers receives a distinct configuration derived from its position in the topology graph—no two devices share the same output.

19.7 Stage 6: Atomic write

The `TransactionalOutput` writer collects all files in temporary locations, then atomically renames them via `POSIX rename(2)`. Per device, three files are written:

- `site-a-r1.conf`: the rendered configuration
- `site-a-r1.deviceir.json`: the raw stanza list (audit trail)
- `site-a-r1.lineage.json`: per-line provenance mapping

If any stage fails, all temporary files are dropped—no partial output directory is ever visible.

20 Error Model

NETCFG uses typed error enums with `thiserror` derivation and `miette` diagnostic annotations. Table 38 summarises the error types.

Table 38. Error enums and their variants.

Enum	Variants	Context
ParseError	YamlSyntax, Validation, Deserialize, LayerOrdering	design plan parsing
SelectorError	InvalidSyntax, Compile, Evaluate, NonBooleanResult, Polars, Json, TopologyAccess	Selector compilation and evaluation
PrimitiveError	Validation, Execute, Selector, Topology, Graph, Polars, Json	Primitive execution
ExecuteError	Parse, Primitive, Shadow, Plan, Assertion	Top-level executor
ShadowError	Clone, Commit, Validation	Shadow topology lifecycle
AssertionError	Failed, Selector	Assertion evaluation
DiffError	TopologyAccess, ParseError	Configuration diffing
RenderError	Io, Jinja, Json, MissingTemplate	Template rendering

ExecuteError variants carry `miette::Diagnostic` annotations with diagnostic codes (e.g., `netcfg::parse`, `netcfg::primitive::execution_failed`, `netcfg::assertion`) enabling structured error handling in CI pipelines.

`ParseError::YamlSyntax` includes a source span (`miette::SourceSpan`) pointing at the offending line, enabling IDE-quality error rendering.

21 Editor and Service Integration

21.1 Language Server (LSP)

The `netcfg-lsp` crate provides a Language Server Protocol implementation built on `tower-lsp` and `Tokio`. It communicates via `stdin/stdout` and offers three capabilities:

1. **Real-time diagnostics.** On file open, change, and save, the server parses the design plan and maps errors to LSP diagnostics with line-column precision. Text parse errors (which embed location as free text) are converted to line/column positions via pattern matching. Diagnostics are published with source label `ank_netcfg` and severity `ERROR`.
2. **Primitive auto-completion.** When the cursor follows a `type:` token (detected by a suffix heuristic), the server offers completion items for all twenty-three primitives, each with a one-line docstring. Trigger characters are space and colon.
3. **Document synchronisation.** Open documents are tracked in a `DashMap<String, String>` (lock-free concurrent map). Full-text sync (`TextDocumentSyncKind::FULL`) is used for correctness. On close, the document is removed and diagnostics are cleared.

21.2 REST API Microservice

The `netcfg-api` crate exposes the compiler as a stateless HTTP service via `axum`, enabling CI/CD pipelines and enterprise orchestrators to compile design plans without a local `netcfg` binary.

Endpoint: `POST /compile` (port 3000).

Table 39. Compile endpoint request and response schema.

Direction	Field	Type	Description
Request	topology_zip_base64	String	Base64-encoded .nnt archive
Request	design_plan_yaml	String	Raw YAML design plan
Response	status	String	success or error
Response	configs	Map<String, String>	Hostname → rendered config
Response	device_ir	Map<String, JSON>	Hostname → DeviceContext IR
Response	warnings	Vec<String>	Compilation warnings

The service executes in a strict sandbox mode: `inject_secrets` and `wasm_plugin` primitives are disabled by default to prevent remote exfiltration of environment variables. The compilation pipeline decodes the base64 payload into a temporary file, loads the topology, parses the design plan, executes via `Executor::apply()`, and extracts per-device IR. All processing is in-memory with no persistent state; the service scales horizontally behind a load balancer.

22 Limitations

22.1 edge_properties not applied (mesh_nodes)

Description. The `MeshNodesSpec.edge_properties` field is accepted by the design plan parser but intentionally not applied. Applying it requires an edge `DataFrame` API on `nnt_topology::Topology` equivalent to `set_dataframe/update_dataframe` but keyed on `(source_id, target_id)` pairs.

Workaround. Edge properties can be encoded as node metadata on both endpoints (e.g., `link_bandwidth` as a per-node field keyed by peer ID).

Planned resolution. Awaiting `nnt-topology` ≥ 0.2 edge `DataFrame` support. A tracking test (`edge_properties_deferred_until_nnt_edge_dataframe`) is present in the test suite.

22.2 Mapping AST inspection not implemented

Description. The `inspect -ast` command can parse and display design plan ASTs but does not yet support mapping documents. Attempting to inspect a mapping file prints “Mapping AST visualization coming soon.”

Workaround. Inspect the mapping YAML directly. The format is simple (a `rules` list with `selector` and `stanza` fields).

Planned resolution. When the mapping DSL grows beyond simple `selector/stanza` pairs, AST visualisation will be added.

22.3 Shadow validation is a placeholder

Description. The `ShadowTopology::validate()` method currently only checks that the shadow did not become empty. It does not perform structural integrity checks (dangling edges, orphaned protocol nodes, etc.).

Workaround. Design-rule assertions can catch many structural problems. Use `min_edges` and `field_exists` assertions as guardrails.

Planned resolution. Integrate with the NTE policy validation framework when available.

22.4 No intra-primitive rollback

Description. If a primitive fails partway through execution (e.g., `provision_ips` exhausts its pool after allocating half the edges), the shadow topology retains the partial mutations. The pipeline aborts (returning an error), and the shadow is discarded, but there is no mechanism to roll back individual primitives within a shadow.

Workaround. This is usually not a problem: the entire shadow is discarded on error, so no partial state reaches production. The limitation matters only for debugging: the error message may not reflect the full extent of partial mutations.

Planned resolution. No current plan. Intra-primitive rollback would require snapshotting the shadow before each primitive, which doubles memory usage.

22.5 generate CLI bypasses mapping document

Description. The generate CLI command renders configs directly from DeviceContext via templates, bypassing the mapping document. The full pipeline (with mapping evaluation and DeviceIR) is implemented in `config_gen::generate_configs()` but the CLI does not expose it yet.

Workaround. Use the `generate_configs()` function directly from Rust code for the full pipeline with mapping support.

Planned resolution. Add `-mapping` flag to the generate CLI command.

22.6 No incremental compilation

Description. Every invocation recompiles the entire topology from scratch. The diff engine can identify which devices changed, but the pipeline cannot skip unchanged devices during compilation.

Workaround. Use `ci-run -json` to compute the diff, then selectively push only the changed configuration files.

Planned resolution. Under investigation. Incremental compilation requires caching intermediate state (the desired topology) between runs and detecting which design plan changes invalidate which nodes.

22.7 No runtime verification or drift detection

Description. NETCFG is a generation-time tool. It produces configuration artifacts but does not push them to devices, monitor compliance, or detect post-deployment configuration drift. Closed-loop verification requires integration with external tools.

Workaround. Periodically regenerate configurations from the canonical design plan and diff against deployed state using `ci-run -json`. Use Batfish [10] or gNMI-based telemetry for runtime compliance checking.

22.8 Shadow topology cloning cost

Description. The transactional shadow is created via a JSON serialisation round-trip, which is $O(n)$ in topology size. For 10,000-node topologies this adds ~200 ms of overhead.

Workaround. Not typically a bottleneck, but for interactive workflows the cost is noticeable. Future work may explore copy-on-write graph snapshots in the NTE engine.

22.9 WASM plugin is experimental

Description. The `wasm_plugin` primitive validates module compilation and symbol resolution but does not implement full linear memory I/O for JSON data exchange between the Rust host and WASM guest. The current implementation is proof-of-concept only.

Workaround. Use `custom_primitive` with nested built-in primitives for extensibility. For complex transformations, implement them as Rust primitives.

Planned resolution. Full C-ABI memory allocation between host and guest is required for production use. See §24.

22.10 Mapping document limitations

Description. Two limitations affect the mapping workflow: (1) the `inspect -ast` command does not yet support mapping documents (§22.2); and (2) there is no auto-generation of mapping rules from OpenConfig YANG schemas or device inventory data—operators must author the mapping document by hand, which requires knowledge of DeviceContext field names and stanza structure.

Workaround. The mapping format is intentionally simple (a rules list with selector and stanza fields). Example mappings ship with the worked examples (`three-site-mesh-mapping.yaml`, `evpn-vxlan-mapping.yaml`).

22.11 NSoT push not implemented (`netcfg sync push`)

Description. The `netcfg sync push` subcommand is defined in the CLI but returns an error when invoked. Pushing intent back to an NSoT system requires a write-capable provider interface (e.g. NetBox REST API mutations) that has not yet been implemented.

Workaround. Export the compiled topology to an NTE archive (`sync pull -o`) and ingest it into the NSoT manually.

Planned resolution. A `NetBoxWriter` provider will be added alongside the existing `NetBoxProvider` read path.

23 Conclusion

Network configuration has historically been treated as a text templating problem, leading to fragile automation pipelines where physical inventory, logical intent, and vendor syntax are deeply entangled. NETCFG demonstrates that treating network configuration as a strict *compilation problem* provides a far more stable foundation.

By modeling the network as a multi-layered topology graph, NETCFG allows operators to write thin, declarative Architectural Plans that project physical inventory into logical overlays (e.g., BGP, OSPF). The introduction of the NETCFG Query Language (NQL) enables powerful, cross-layer assertions that validate structural intent *before* any device syntax is generated. Finally, by extracting a strict Vendor-Neutral Intermediate Representation (DeviceIR), the compiler decouples architectural design rules from physical hardware constraints, making multi-vendor operations deterministic.

24 Future Work

Autonomous Operations & Telemetry

- **Automated blast-radius calculation:** Extending the impact analysis framework to actively traverse topological dependencies and quantify indirect operational risks before deployment.
- **Continuous telemetry ingestion:** Integrating runtime gNMI telemetry ingestion for configuration drift detection and reporting against the expected DeviceIR state.

Performance and Scale

- **GPU-accelerated primitive execution:** Offloading NETCFG query evaluation and IPAM allocation to the GPU via NTE's WebGPU compute backend, targeting 100K+ node topologies.
- **Incremental compilation:** Caching intermediate state to avoid recompiling unchanged portions of the topology.

Developer Experience

- **Semantic cross-reference diagnostics:** Expanding the `nte-lsp` daemon to validate semantic cross-references (group names, zones, objects) and surface diagnostic errors in real-time within editor clients.
- **LSP assertion integration:** Extending the language server to evaluate design-rule assertions in real time, providing inline warnings for query violations and IP pool capacity.
- **Mapping CLI integration:** Exposing the full pipeline (including mapping evaluation) through the `generate` CLI command.
- **Annotation defaults and inheritance:** Currently, every node must carry its own annotations (e.g. `device_os: eos` on each device). A default/inheritance mechanism—for example, site-level or group-level defaults that individual nodes can override—would reduce repetition in large topologies and reinforce the “topology as source code” model.

References

- [1] S. Shenker, “The future of networking, and the past of protocols,” in *Open Networking Summit*, vol. 20, Palo Alto, CA: Open Networking Foundation, 2011, pp. 1–30.
- [2] P. Gill, N. Jain, and N. Nagappan, “Understanding network failures in data centers: Measurement, analysis, and implications,” in *ACM SIGCOMM computer communication review*, vol. 41, New York, NY, USA: ACM, 2011, pp. 350–361.
- [3] J. Zhu et al., “MALT: Multi-abstraction layer topology,” in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, Santa Clara, CA: USENIX Association, 2020, pp. 313–328.
- [4] OpenConfig Working Group, *OpenConfig*, <https://openconfig.net>, 2023.
- [5] M. Bjorklund, “The YANG 1.1 data modeling language,” IETF, RFC 7950, 2016.
- [6] NetBox Community, *NetBox: The source of truth for network automation*, <https://netbox.dev/>, 2023.
- [7] Network to Code, *Nautobot: Network source of truth and automation platform*, <https://nautobot.com/>, 2023.
- [8] Red Hat, *Ansible automation platform*, <https://www.ansible.com/>, 2023.
- [9] Nornir Automation, *Nornir: Pluggable multi-threaded framework with inventory management to help operate collections of devices*, <https://nornir.tech/>, 2023.
- [10] Intentionet, *Batfish: Network configuration analysis tool*, <https://www.batfish.org/>, 2023.
- [11] C. Lattner and V. Adve, “LLVM: A compilation framework for lifelong program analysis & transformation,” in *International Symposium on Code Generation and Optimization, 2004. CGO 2004.*, IEEE, San Jose, CA: IEEE, 2004, pp. 75–86.
- [12] Google, *Common Expression Language (CEL)*, <https://github.com/google/cel-spec>, 2023.
- [13] S. Knight, *AutoNetKit NTE: The network topology engine*, 2026.
- [14] N. Foster et al., “Frenetic: A network programming language,” in *Proceedings of the 16th ACM SIGPLAN International Conference on Functional Programming*, 2011, pp. 279–291.
- [15] C. J. Anderson et al., “NetKAT: Semantic foundations for networks,” in *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 2014, pp. 113–126.
- [16] R. Beckett, R. Mahajan, T. Millstein, J. Padhye, and D. Walker, “Propane: A general approach to network configuration synthesis,” in *Proceedings of the 2016 ACM SIGCOMM Conference*, 2016, pp. 440–453.
- [17] A. El-Hassany et al., “Network configuration synthesis with logic-based routing models,” in *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2017.
- [18] P. Bosshart et al., “P4: Programming protocol-independent packet processors,” in *ACM SIGCOMM Computer Communication Review*, vol. 44, ACM New York, NY, USA, 2014, pp. 87–95.
- [19] P. Kazemian, G. Varghese, and N. McKeown, “Header space analysis: Static checking for networks,” in *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, 2012, pp. 113–126.
- [20] A. Fogel et al., “A general approach to network configuration analysis,” in *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, 2015, pp. 469–483.
- [21] R. Beckett, A. Gupta, R. Mahajan, and D. Walker, “A general approach to network configuration verification,” in *Proceedings of the 2017 Conference of the ACM Special Interest Group on Data Communication*, 2017, pp. 155–168.
- [22] R. Birkner et al., “Config2spec: Mining network specifications from network configurations,” in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, 2020.
- [23] Forward Networks, *Forward Enterprise*, <https://www.forwardnetworks.com/>, 2024.
- [24] SaltStack, *Salt: Configuration management and orchestration*, <https://saltproject.io/>, 2024.
- [25] Cisco Systems, *Cisco Network Services Orchestrator (NSO)*, <https://www.cisco.com/c/en/us/products/cloud-systems-management/network-services-orchestrator/index.html>, 2023.

- [26] I. Pepelnjak, *Netlab: Network configuration and testing lab*, <https://netlab.tools/>, 2024.
- [27] Bytecode Alliance, *Wasmtime: A fast and secure runtime for WebAssembly*, <https://wasmtime.dev/>, 2024.